

TDK-dolgozat

KCMamba: Kalman-vezérelt szelektív
állapottér-modell robusztus
idősor-előrejelzésre

Kovács András

programtervező informatikus BSc, III. évfolyam

Eötvös Loránd Tudományegyetem
Informatikai Kar

Budapest, 2026. május



EÖTVÖS LORÁND TUDOMÁNYEGYETEM

INFORMATIKAI KAR

MESTERSÉGES INTELLIGENCIA TANSZÉK

KCMamba: Kalman-vezérelt szelektív
állapottér-modell robusztus
idősor-előrejelzésre

Témavezető:

Kristian Fenech Arian

oktató

Szerző:

Kovács András

programtervező informatikus BSc, III. évfolyam

Budapest, 2026. május

Kivonat

Ez a szakdolgozat a **KCMamba** (Kalman Core-Attention Mamba) architektúrát mutatja be, amely differenciálható Kalman-szűrési mechanizmust integrál a Mamba szelektív állapottér-modellbe (SSM) a zajrobosztus idősor-előrejelzés érdekében. A javasolt architektúra tanult Q_{net} -, R_{net} - és K_{net} -alhálózatokat alkalmaz, amelyek adaptívan szabályozzák a mérési és folyamat zaj arányát, ezáltal a modell zajmentes adatokon passzív előrejelzőként, zajosított bemeneten aktív szűrőként viselkedik.

Empirikus értékelést végzünk három nyilvános LTSF-adatkészleten (Exchange-Rate, ETTh1, ETTh2), szintetikus zajosított ($\sigma \in \{0, 1, 3, 5\}$) és alacsony adatbőségű ($\text{stride} \in \{1, 16\}$) körülmények között.

Az Exchange-Rate adatkészleten végzett 3-mag kísérletben ($s = 16$) a KCMamba átlagos MSE-degradációja $\sigma=0$ -tól $\sigma=5$ -ig +1 528%, a Fair Mambáé +28 234%, ami $18,5\times$ -os stabilitási különbséget jelent; a legsúlyosabb konfigurációban ($s=16, \sigma=5$) a KCMamba MSE-je 1,41, a Fair Mambáé 22,72. Az ablációs vizsgálat kimutatta, hogy a tanult K -gain elhagyása a degradációt $2,4\times$ -ra növeli, ezzel azonosítva a $Q_{\text{net}}/R_{\text{net}}$ alhálózatokat mint a robusztusság elsődleges forrását.

A szakdolgozat a teljes szoftverrendszer leírását is tartalmazza: egy aszinkron, eseményvezérelt kereskedési motort hexagonális architektúrában, visszatesztelő ke-retrendszert és élő Binance-kereskedési adaptert.

Abstract

This thesis presents **KCMamba** (Kalman Core-Attention Mamba), a hybrid neural architecture that integrates a differentiable Kalman filtering mechanism into the Mamba selective state-space model (SSM) for noise-robust time series forecasting. The proposed architecture employs learned Q_{net} , R_{net} , and K_{net} subnetworks that adaptively regulate the ratio of process and measurement noise, enabling the model to behave as a passive predictor on clean data and as an active filter under noisy conditions.

We conduct empirical evaluation on three public LTSF benchmarks (Exchange-Rate, ETTh1, ETTh2) under synthetically injected noise ($\sigma \in \{0, 1, 3, 5\}$) and low data-availability regimes (stride $\in \{1, 16\}$).

In the Exchange-Rate benchmark ($s = 16$, 3-seed experiment), KCMamba’s average MSE degradation from $\sigma=0$ to $\sigma=5$ is +1 528%, compared to +28 234% for Fair Mamba, an $18.5\times$ stability gap; in the hardest configuration ($s=16$, $\sigma=5$) KCMamba achieves MSE 1.41 versus 22.72 for Fair Mamba. The ablation study identifies the $Q_{\text{net}}/R_{\text{net}}$ subnetworks as the primary source of robustness: removing the learned K -gain increases degradation by $2.4\times$.

The thesis also describes the complete production software system: an asynchronous, event-driven trading engine built in a hexagonal architecture, a backtesting framework, and a live Binance trading adapter.

Tartalomjegyzék

Kivonat	1
Abstract	2
1. Bevezetés	6
1.1. Motiváció	6
1.2. Célkitűzések	7
1.3. Hozzájárulások	7
1.4. Dolgozat felépítése	7
2. Irodalmi háttér	8
2.1. Kalman-szűrő és általánosításai	8
2.1.1. Tanult Kalman-szűrők	9
2.2. Állapottér-modellek	9
2.2.1. S4	9
2.2.2. Mamba	10
2.2.3. Fair Mamba	10
2.3. Hosszú távú idősor-előrejelzés	11
2.3.1. Problémaformulázás	11
2.3.2. Referenciamodellek	11
2.3.3. Adatkészletek	12
2.4. Zajrobustusság	12
3. A KCMamba architektúra	14
3.1. Áttekintés	14
3.2. KCMamba blokk	16
3.2.1. Bemeneti projekció és 1D-konvolúció	16
3.2.2. Kalman-paraméter hálózatok	17
3.2.3. Kalman-gain számítás	18

3.2.4.	Állapotfrissítés párhuzamos scan-nel	18
3.2.5.	Összegzés és reziduál kapcsolat	19
3.3.	KCMamba verem	20
3.4.	Paraméterkomplexitás	23
3.5.	Összehasonlítás a standard Mamba-val	24
3.6.	Rendszerarchitektúra	25
3.7.	Modul- és osztályszerkezet	25
4.	Kísérletek és értékelés	27
4.1.	Kísérleti beállítások	27
4.1.1.	Hardver és szoftver	27
4.1.2.	Adatkészletek	27
4.1.3.	Zajmodell	27
4.1.4.	Modellkonfiguráció és tanítási hiperparaméterek	28
4.2.	Eredmények	28
4.2.1.	Exchange-Rate adatkészlet	28
4.2.2.	ETTh1 adatkészlet	29
4.2.3.	ETTh2 adatkészlet	30
4.2.4.	Összefoglaló statisztika	30
4.3.	A Kalman-gain dinamikája	31
4.4.	Diszkusszió	34
4.5.	Hosszú szekvenciák vizsgálata	35
4.6.	Stride $\times$ Hosszú Szekvencia	36
4.7.	Ablációs vizsgálat és statisztikai szignifikancia	38
5.	Összegzés	40
5.1.	Főbb eredmények	40
5.2.	Korlátok	41
5.3.	Jövőbeli irányok	42
	Köszönetnyilvánítás	43
	A saját eredmények lehatárolása	44
	A. Mesterséges intelligencia alapú eszközök felhasználása	45

Irodalomjegyzék	45
Ábrajegyzék	48

1. fejezet

Bevezetés

Az idősor-előrejelzés a gépi tanulás egyik kiterjedten kutatott részterülete, amelynek gyakorlati jelentőségét a valós adatok szinte kivétel nélkül jelen lévő zajossága adja. Szenzorkiesések, aggregációs torzítások és piaci mikrostruktúra-zaj egyaránt rontják a jel/zaj arányt, amelyre a jelenlegi modellek érzékenynek bizonyulnak. A hosszú távú idősor-előrejelzés (LTSF) feladata ennél is nehezebb. Adott L hosszúságú előzmény alapján kell becsülni a következő H lépést, ahol H nagyságrendileg elérheti vagy meghaladhatja L -et.

1.1. Motiváció

A modern szekvencia-model-családok, a Mamba-alapú szelektív állapottér-modellek [1, 2] és a klasszikus rekurrens hálózatok [3] az adatminőségi problémát közvetett általánosítás révén kezelik, vagyis a zaj- és jelkomponens szétválasztását kizárólag az adatvezérelt reprezentációtanulásra bízzák. Zajmentes adatokon ez az eljárás kielégítően teljesít, ám a megnövelt zajszintre való általánosítóképesség a mérések alapján nem garantált.

A Kalman-szűrő [4] ezzel szemben *explicit* statisztikai modellt állít fel a folyamat-zajra (Q) és a mérési zajra (R), és ezek arányából származtat egy K gain együtthatót, amely az előzmény-állapot és az aktuális megfigyelés közötti optimális súlyozást határozza meg. Ennek az induktív torzításnak egy differenciálható neurális hálózatba történő beépítése elméleti szempontból ígéretes megközelítés a zajrobustusság növelésére.

Ezen megfontolásra épül a jelen szakdolgozatban bemutatott **KCMamba** architektúra.

1.2. Célkitűzések

Három kérdést vizsgálunk: (1) versenyképes-e a KCMamba tiszta adatokon a standard LTSF modellekkel szemben; (2) mekkora MSE-degradációt szenvednek el a modellek szintetikus zajosított bemeneten ($\sigma \in \{0, 1, 3, 5\}$); és (3) kevés trainingablak ($\text{stride} \in \{1, 16\}$) esetén melyik architektúra generalizál jobban.

1.3. Hozzájárulások

A dolgozat öt konkrét hozzájárulást tartalmaz. A **KCMamba** architektúra tanult Q_{net} , R_{net} és K_{net} alhálózatokat ágyaz egy differenciálható Kalman-szűrőbe, a rekurzió kiértékelését párhuzamos prefix-scan alapú GPU-implementációval gyorsítva. Mérésekkel alátámasztjuk, hogy a tanult K valóban adaptívan változik a zajszinttel: $\sigma = 0$ -nál $K \approx 0,71$ (passzív átengedés), $\sigma = 5$ -nél $K \approx 0,07$ (erős szűrés).

Az összehasonlítás három adatkészleten (Exchange-Rate, ETTh1, ETTh2) megmutatja, hogy a Fair Mamba átlag 28 234% MSE-degradációt mutat zajjal, a KCMamba 1 528%-ot ($18,5\times$ kisebb romlás). Hosszú szekvencia vizsgálatban ($T \in \{96, 192, 336, 512\}$, ETTm1 és Exchange) a KCMamba $T=336$, $\sigma=1$ konfigurációban 31,7%-kal jobb MSE-t ér el az LSTM-nél (0,502 vs 0,735); $T=512$, $\sigma=3$ -nál a KCMamba 0,773, a Fair Mamba 2,121. A $\text{stride} \times$ szekvenciahossz vizsgálatban ($s \in \{1, 16\}$, $T \in \{192, 336, 512\}$) a $s=16$, $\sigma \geq 3$ tartományban a Fair Mamba összomló teljesítményt mutat (8,1–8,5 MSE), míg a KCMamba 1,1–1,3 között marad. A kísérleteket nyílt forráskódú Python/PyTorch/CUDA implementáció kíséri, amelyet egy aszinkron kereskedési rendszerbe integráltunk.

1.4. Dolgozat felépítése

A 2. fejezet összefoglalja a Kalman-szűrő, az SSM-ek és az LTSF-kutatás idevágó irodalmát. A 3. fejezet ismerteti a KCMamba architektúrát és matematikai hátterét. A 4. fejezetben a kísérleti beállítás és az eredmények szerepelnek. A ?? fejezet a megvalósított szoftverrendszert tárgyalja. Az 5. fejezet levonja a következtetéseket.

2. fejezet

Irodalmi háttér

2.1. Kalman-szűrő és általánosításai

A Kalman-szűrő [4] minimális varianciájú, torzítatlan, lineáris becslő olyan rendszerekre, amelyek $x_t \in \mathbb{R}^n$ állapota Gauss-zajjal perturbált lineáris dinamikai modell szerint fejlődik:

$$x_t = Fx_{t-1} + w_t, \quad w_t \sim \mathcal{N}(0, Q), \quad (2.1)$$

$$y_t = Hx_t + v_t, \quad v_t \sim \mathcal{N}(0, R), \quad (2.2)$$

ahol F az állapotátmeneti mátrix, H a mérési mátrix, $Q \in \mathbb{R}^{n \times n}$ a folyamatzaj-kovariancia, és $R \in \mathbb{R}^{m \times m}$ a mérési-zaj-kovariancia. A szűrő két felváltva ismétlődő fázisban működik.

Az *előrejelzési lépésben* (időfrissítés), mielőtt a y_t mérés megérkezne, a szűrő az állapotbecslést és a hibakovarianciát előrevetíti:

$$\hat{x}_{t|t-1} = F\hat{x}_{t-1|t-1}, \quad (2.3)$$

$$P_{t|t-1} = FP_{t-1|t-1}F^\top + Q, \quad (2.4)$$

ahol $P_{t|t-1} \in \mathbb{R}^{n \times n}$ az előrejelzett hibakovariancia. Nagy $P_{t|t-1}$ értéke magas előrejelzési bizonytalanságot jelez. Nagy Q azt jelenti, hogy az állapot gyorsan és kiszámíthatatlanul változik.

A *korrekciós lépésben* (mérésfrissítés), amint a y_t megfigyelés megérkezik, a szűrő az *innováció* $e_t = y_t - H\hat{x}_{t|t-1}$ alapján korrigálja becslését, a Kalman-gain K_t által

súlyozva:

$$K_t = P_{t|t-1} H^\top (H P_{t|t-1} H^\top + R)^{-1}, \quad (2.5)$$

$$\hat{x}_{t|t} = \hat{x}_{t|t-1} + K_t e_t, \quad (2.6)$$

$$P_{t|t} = (I - K_t H) P_{t|t-1}. \quad (2.7)$$

A (2.5). egyenlet a zaj-jel-egyensúly alapvető összefüggését tükrözi. Ha az R mérési zaj nagy az előrejelzett $H P_{t|t-1} H^\top$ kovarianciához képest, a nevező dominál és $K_t \rightarrow 0$: a szűrő nem bíz a mérésben, és az előzmény-állapotra támaszkodik. Ha $R \rightarrow 0$, akkor $K_t \rightarrow H^{-1}$, és a megfigyelés majdnem teljesen felülírja az állapotot.

A skaláris esetben ($F = H = 1$) a gain egyszerűsödik:

$$K_t = \frac{P_{t|t-1}}{P_{t|t-1} + R}. \quad (2.8)$$

Stacionárius feltételek mellett a $P_{t|t-1}$ hibakovariancia egy fix $\bar{P}$ értékre konvergál, és az egyensúlyi gain kizárólag a Q/R aránytól függ: magas folyamat zaj (Q) a K -t 1 felé tolja (bízunk a mérésben), alacsony Q/R arány K -t 0 felé tolja (bízunk a modellben). (3.2.3. szakasz).

2.1.1. Tanult Kalman-szűrők

A neurális Kalman-irodalomban a F, H, Q, R mátrixokat neurális hálózatokkal paraméterezik [5, 6]. A *Deep Kalman Filter* [5] nemlineáris átmenetet enged meg, a *Kalman Variational Autoencoder* [6] a latens állapotok szétválasztott reprezentációját tanítja. A KCMamba ezeken az elveken épít, de nem autoencoderként, hanem szekvencia-előrejelzőként.

2.2. Állapottér-modellek

2.2.1. S4

Az S4 (Structured State Spaces for Sequences) [1] egy folytonos-idejű állapottér-modellből indul ki, amelyben egy d -dimenziós bemeneti sorozatot egy N -dimenziós

látens dinamikán keresztül modellez:

$$h'(t) = Ah(t) + Bu(t), \quad (2.9)$$

$$y(t) = Ch(t), \quad (2.10)$$

ahol $A \in \mathbb{R}^{N \times N}$ az állapotátmeneti mátrix, $B \in \mathbb{R}^{N \times d}$ a bemeneti projekció, $C \in \mathbb{R}^{d \times N}$ a kimeneti projekció, és $h(t) \in \mathbb{R}^N$ a látens állapot. Diszkretizálva egy Δ időlépéssel:

$$h_t = \bar{A} h_{t-1} + \bar{B} u_t, \quad y_t = C h_t, \quad (2.11)$$

ahol $\bar{A} = e^{\Delta A}$ és $\bar{B} \approx (\Delta A)^{-1}(e^{\Delta A} - I)\Delta B$. Az S4 kulcsinnovációja az A mátrix HiPPO-inicializációja, amely az állapotnak polinomiális bázisú hosszú távú memóriát biztosít. A konvolúciós nézet, $\bar{K} = (C\bar{B}, C\bar{A}\bar{B}, \dots, C\bar{A}^{T-1}\bar{B})$ kernel, lehetővé teszi, hogy a tanítás $O(T \log T)$ -ben gyors Fourier-transzformációval (FFT) végezhető, míg az inferencia a rekurrens formában $O(1)$ memóriával fut lépésenként.

2.2.2. Mamba

A Mamba [2] az S4 alapjaira építve bevezeti az *input-függő szelektivitást*: míg az S4-ben az A , B , C mátrixok rögzítettek a teljes szekvenciára, a Mamba ezeket az aktuális bemenettől függővé teszi. Ez a szelektivitás a lényege: a modell minden időlépésben eldönti, mit őrizzen meg az állapotban és mit felejtse el. A bemeneti projekciós mátrix $B(u_t) \in \mathbb{R}^{N \times d}$, a kimeneti projekció $C(u_t) \in \mathbb{R}^{d \times N}$ és az időléptékező $\Delta(u_t) \in \mathbb{R}^d$ mindegyike az aktuális $u_t \in \mathbb{R}^d$ bemeneti vektortól függ:

$$h_t = \bar{A}(u_t) h_{t-1} + \bar{B}(u_t) u_t, \quad y_t = C(u_t) h_t, \quad (2.12)$$

ahol $h_t \in \mathbb{R}^N$ a látens állapotvektor (N az állapotméret), $\bar{A}(u_t) = e^{\Delta(u_t)A}$ és $\bar{B}(u_t) \approx \Delta(u_t)B$ a Δ alapján diszkretizált mátrixok. A modell így szelektíven dönti el, hogy a bemeneti sorozat melyik részét tárolja tartósan az állapotban, és melyiket hagyja figyelmen kívül. Hardver-barát párhuzamos prefix-scan implementációval $O(T)$ memória- és időkomplexitást ér el.

2.2.3. Fair Mamba

Az összehasonlítási alap, amelyet *Fair Mamba* névvel illetünk, egy csatornánkénti (*channel-wise*) Mamba, amelyet a KCMamba-éval azonos hiperparaméterekkel és

tanítási protokoll szerint tanítunk. A *fair* jelző arra utal, hogy a paraméterbudget és az adatmennyiség azonos, így az eredménybeli különbség kizárólag az architektúrából fakad.

2.3. Hosszú távú idősor-előrejelzés

2.3.1. Problémaformulázás

A hosszú távú idősor-előrejelzés (LTSF) egy $\mathbf{X} = (x_1, \dots, x_L) \in \mathbb{R}^{L \times d}$ bemeneti ablakot tekint, amelynek L historikus lépése d csatornán van megadva, és $\hat{\mathbf{X}} = (\hat{x}_{L+1}, \dots, \hat{x}_{L+H}) \in \mathbb{R}^{H \times d}$ előrejelzését kívánja adni a következő H lépésre. A cél a normált átlagos négyzetes hiba minimalizálása:

$$\text{MSE} = \frac{1}{Hd} \sum_{h=1}^H \sum_{j=1}^d \frac{(x_{L+h,j} - \hat{x}_{L+h,j})^2}{\text{Var}(x_{\cdot,j})}, \quad (2.13)$$

ahol a csatornánkénti varianciával való normálás különböző skálájú adatkészletek eredményeit összehasonlíthatóvá teszi. A *hosszú távú* jelző a $H \geq L$, illetve H és L hasonló nagyságrendű esetét jelöli: az ilyen feladatban a modellnek a visszatekintési ablakán jócskán túlra kell előrejelezni, ez pedig lényegesen nehezebb, mint a rövid távú eset, ahol $H \ll L$.

2.3.2. Referenciamodellek

Informer [7] volt az első, kifejezetten hosszú távú előrejelzésre tervezett Transformer-változat. A standard önfigyelés $O(L^2)$ idő- és memóriakomplexitása hosszú bemeneti ablakoknál elfogadhatatlanul magas. Az *Informer* ezt *ProbSparse attention*-tel orvosolja: KL-divergencia-bebecslés alapján azonosítja az $O(\log L)$ leginformatívabb lekérdező pozíciót, ezzel $O(L \log L)$ -re csökkenti a komplexitást. Disztillációs művelet felezi a szekvenciahosszat az enkoderblokkok között, így több ezer lépéses visszatekintési ablakok is kezelhetők.

Autoformer [8] minden sorozatot egy trendkomponensre (mozgóátlag-kernellel becsülve) és egy szezonális maradéktagra bont, és a Transformatort kizárólag a maradéktagokra alkalmazza. Pontszorzat-figyelem helyett *Auto-Korrelációs* mechanizmust használ: megkeresi azt a τ késleltetést, amely maximalizálja a lekérdező jel autokorrelációját, majd ezeken a késleltetési pozíciókon összesíti az értékeket. A

mechanizmus FFT-vel $O(L \log L)$ komplexitású, és periodikus jelekre illeszkedő induktív torzítással rendelkezik.

DLinear [9] elveti az architektúráis bonyolultságot, és az előrejelzést minden visszatekintési ablakra egyetlen lineáris réteggel állítja elő, opcionálisan trend-szezonális felbontás után. Egyszerűsége ellenére a DLinear számos Transformer-változattal azonos szinten teljesített, sőt azokat felülmúlta szabványos benchmarkokon, bizonyítva, hogy az architektúráis bonyolultság nem feltétele a jó előrejelzési teljesítménynek. Referenciamodellként releváns marad, mert minden új modellnek igazolnia kell, hogy a magasabb számítási ráfordítás indokolt ehhez az egyszerű alapmodellhez képest.

2.3.3. Adatkészletek

Az Exchange-Rate adatkészlet [10] nyolc valutaárfolyamot tartalmaz, amelyeket 1990 és 2016 között napi szinten vettek fel, csatornánként 7 588 időlépéssel. Az árfolyam-adatok dinamikája majdnem egységgyök-jellegű: minden napi változás véletlenszerű bolyongáshoz közelít, szinte elhanyagolható vissza-átlagolással. Ez kemény feladatot jelent az autoregresszív modelleknek, és természetes tesztkörnyezetet jelöl a Kalman-struktúra számára, ahol a jel-zaj arány eleve alacsony.

Az ETTh1 és ETTh2 adatkészletek [7] kínai transzformátorállomásokból gyűjtöttek óránkénti villamosenergia-terhelést és olajhőmérsékletet két éven keresztül (17 420 lépés, $d = 7$ csatorna). Az ETTh1 egyértelmű napi periodicitást mutat, az ETTh2 zajosabb és szabálytalanabb. A két adatkészlet eltérő nehézségű tesztkörnyezetet nyújt a zajrobustusság kiértékeléséhez.

2.4. Zajrobustusság

Az idősor-modellek zajérzékenységének szisztematikus elemzése az LTSF-irodalomban aláreprézantált terület. A PatchTST [11] patch-tokenizálással csökkenti a szomszédos mintavételezési torzítást. Ennek közvetett zajcsökkentő hatása van, bár nem ez az eljárás fő célkitűzése.

A jelen munkában az értékelési protokoll: $\tilde{x}_t = x_t + \varepsilon_t$, ahol $x_t \in \mathbb{R}^d$ az eredeti idősorjel, $\varepsilon_t \sim \mathcal{N}(0, \sigma^2 I)$ additív Gauss-zaj ($\sigma > 0$ a szórás, $I \in \mathbb{R}^{d \times d}$ az egységmátrix), és $\sigma \in \{0, 1, 3, 5\}$ a vizsgált zajszintek halmaza. A tanító- és tesztalmazt egyaránt

azonos σ -val zajosítjuk, tehát nem domain-shift-ről, hanem egyenletes adatminőség-romlásról van szó.

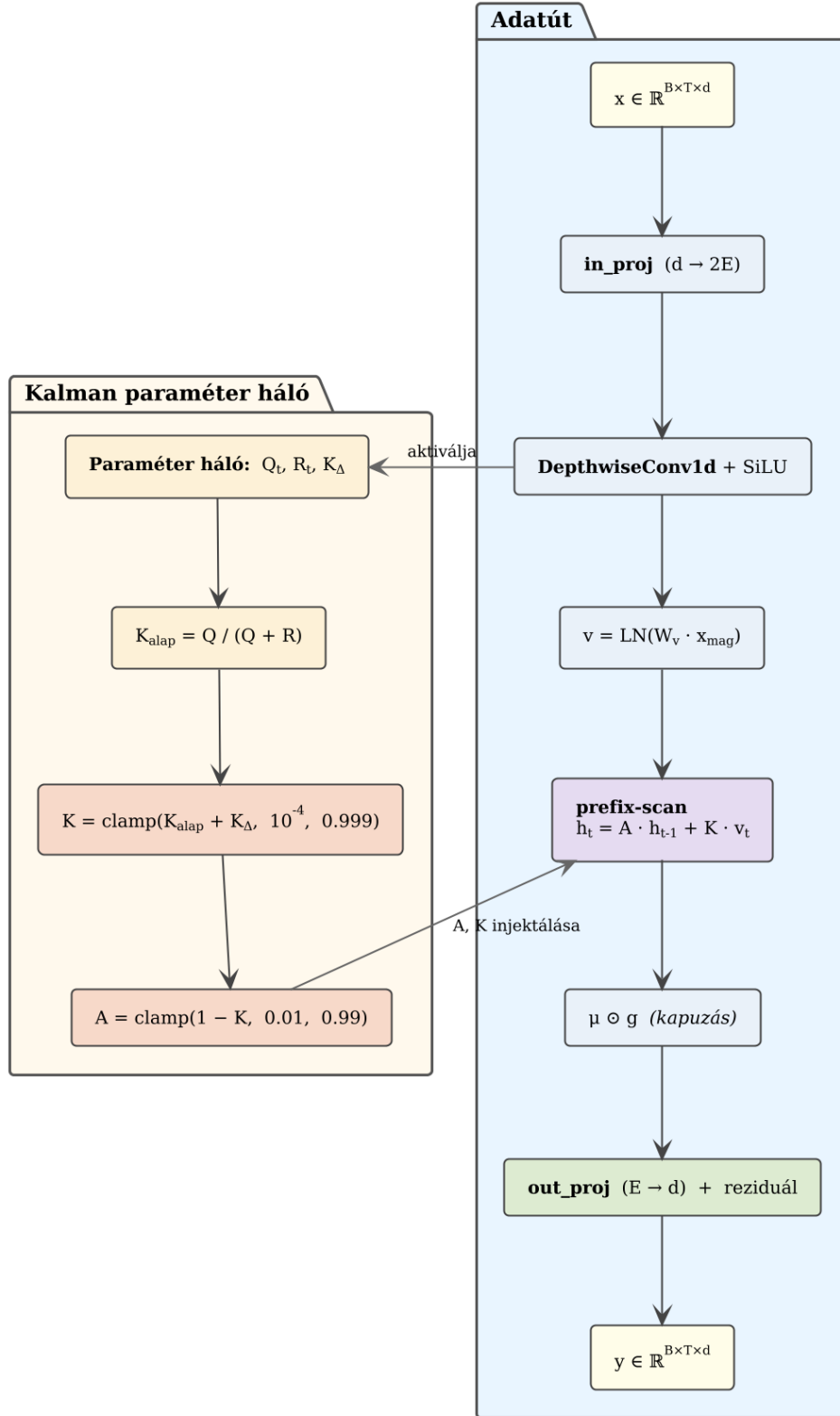
3. fejezet

A KCMamba architektúra

3.1. Áttekintés

A KCMamba (Kalman Core-Attention Mamba) egy hibrid neurális architektúra, amely a Mamba SSM [2] rekurrens dinamikájába egy differenciálható, tartalomfüggő Kalman-szűrő réteget integrál. A modell nem rögzített A felejtési faktorral dolgozik, hanem egy, a bemenetből tanult K -gainből származtatott $A = 1 - K$ együtthatót használ az állapotfrissítésben. Ez lehetővé teszi, hogy az előzmény- állapot és az aktuális megfigyelés közötti súlyozás a zajszintnek megfelelően, adaptívan álljon be.

A modell két szintből épül fel: az alap feldolgozó egység a `KCMambaBlock` (tartalom-függő Kalman-szűrés és állapotfrissítés), fölötte a `KCMambaStack`: N egymás utáni KC-blokk cross-attention réteggel a végén.



3.1. ábra. A KCMamba blokk teljes adatfolyam-diagramja. A sárga blokkok (Q_{net} , R_{net} , K_{net}) tanítják a Kalman-statisztikákat, a lila blokkban történik a párhuzamos prefix-scan GPU-n $O(\log T)$ szinkronizációval. A zöld blokkok a bemeneti és kimeneti projekciókat jelölik.

3.2. KCMamba blokk

Legyen a bemenő szekvencia $x \in \mathbb{R}^{B \times T \times d}$, ahol B a kötegméret, T a szekvenciahossz, és d a jellemzők száma.

3.2.1. Bemeneti projekció és 1D-konvolúció

A bemenetet két részre vetítjük egy lineáris réteggel, majd SiLU aktivációs kapuzást alkalmazunk:

$$[x_{\text{cw}}, x_{\text{gate}}] = W_{\text{in}} x, \quad (3.1)$$

$$\text{gate} = \text{SiLU}(x_{\text{gate}}), \quad (3.2)$$

ahol $W_{\text{in}} \in \mathbb{R}^{2d_{\text{inner}} \times d}$ a bemeneti projekciós mátrix, d_{inner} a belső csatornadi-menzió, x_{cw} a Kalman-szűrési ághoz, x_{gate} a kapuzáshoz továbbított csatorna, és $\text{SiLU}(z) = z \cdot \sigma(z)$ a Sigmoid Linear Unit aktiváció, ahol $\sigma(z) = 1/(1 + e^{-z})$ a logisztikus szigmoid. A kétágú projekció két különböző szerepet tölt be. Az x_{cw} csatorna a Kalman-szűrési ág bemenete, míg az x_{gate} csatorna a kimenet amplitúdójának független modulálásáért felel. Ez a szétválasztás megakadályozza, hogy a kapuzási jel beavatkozzon a Kalman-állapot dinamikájába. A SiLU aktiváció ReLU-val szemben azért előnyös, mert sima és kis negatív értékeket is megenged, ami a korai tanítás során javítja a gradiens-átmenetet.

A globális szekvenciális scan előtt az x_{cw} csatornán egy depthwise 1D-konvolúció ($k = 4$ kernel) biztosítja a lokális kontextus-integrációt:

$$x_{\text{core}} = \text{SiLU}(\text{Conv1D}(x_{\text{cw}})). \quad (3.3)$$

A konvolúció csatornánként egy szűrőt alkalmaz (depthwise), ezért mindössze $k \cdot d_{\text{inner}}$ extra paramétert igényel, miközben rövid-horizontú mintázatokat, például rövid-távú lendületet és trendirányokat, ragad meg, mielőtt a Kalman-szűrő a teljes szekvenciát feldolgozza.

3.2.2. Kalman-paraméter hálózatok

Három kis, kétrétegű, teljesen összekötött alhálózat tanítja meg a Kalman-szűrő statisztikáit. A Q_{net} és R_{net} hálózatok az időlépésenkénti zajvarianciákat becslik:

$$Q_t = \text{softplus}(Q_{\text{net}}(x_{\text{core}})) \cdot \text{softplus}(q_{\text{scale}}), \quad (3.4)$$

$$R_t = \text{softplus}(R_{\text{net}}(x_{\text{core}})) \cdot \text{softplus}(r_{\text{scale}}). \quad (3.5)$$

ahol $Q_t > 0$ a *folyamatzaj* becsült varianciája, vagyis az állapot-dinamikára vonatkozó bizonytalanság, $R_t > 0$ a *mérési zaj* becsült varianciája, vagyis a bemeneti megfigyelés bizonytalansága, $q_{\text{scale}}, r_{\text{scale}} \in \mathbb{R}$ tanítható skalár paraméterek, és $\text{softplus}(z) = \ln(1 + e^z) > 0$ garantálja a pozitivitást. A $q_{\text{scale}} = -2,0$, $r_{\text{scale}} = 1,5$ inicializációk biztosítják, hogy kezdetben $R \gg Q$, vagyis $K_{\text{base}} \approx 0.07$, ami erős szűrési módot jelent alapértelmezésként.

Egy harmadik alhálózat, a K_{net} , szintén kétrétegű, teljesen összekötött Multi-Layer Perceptron, és közvetlen reziduál-korrekción nyújt a végső gainhez, ahogy azt a 3.2.3. szakasz részletezi. A $Q_{\text{net}}/R_{\text{net}}$ pár a lassan változó zajkörnyezetet modellezi, míg a K_{net} azonnali reakcióra képes gyors, tartalomfüggő eseményekre, például hirtelen volatilitásnövekedésre vagy trendfordulóra, amelyekre a zaj-arány becslése még nem alkalmazkodott.

3.2.3. Kalman-gain számítás

Az alap Kalman-gain a folyamat- és mérési zaj *arányából* következik, ami a becslélméleti steady-state Kalman-gain klasszikus képlete:

$$K_{\text{base},t} = \frac{Q_t}{Q_t + R_t + \varepsilon}, \quad (3.6)$$

ahol $\varepsilon = 10^{-6}$ a nullával való osztást megelőző numerikus stabilitási konstans. Ha $Q \gg R$, azaz az állapot gyorsan változik, akkor $K \rightarrow 1$, és a modell szorosan követi az aktuális megfigyelést. Ha $R \gg Q$, azaz a mérés zajos, akkor $K \rightarrow 0$, és az előző állapot dominál. Ez egy fizikailag értelmes, kontrollélméleti elvekre támaszkodó prior.

Ehhez járul egy tartalom-függő korrekció a K_{net} alhálózattól, ami olyan gyors rezsimváltások kezelésére képes, amelyekre a lassan alkalmazkodó Q/R arány még nem reagált:

$$K_t = \text{clamp}\left(K_{\text{base},t} + 0.3(\sigma_{\text{sig}}(K_{\text{net}}(x_{\text{core}})) - 0.5), [10^{-4}, 0,999]\right), \quad (3.7)$$

ahol $\sigma_{\text{sig}}(\cdot) = 1/(1 + e^{-(\cdot)})$ a logisztikus szigmoid függvény, amelynek értékkészlete a $(0, 1)$ nyílt intervallum, a $-0,5$ eltolás nullára centrálja a korrekciót inicializáláskor, és a $0,3$ szorzó $(-0,15, +0,15)$ -re korlátozza a hatókörét. A $[10^{-4}, 0,999]$ -re való vágás két degenerált esetet zár ki. $K_t \approx 0$ befagyasztaná az állapotfrissítést, mivel nem integrálna új információt. $K_t \approx 1$ esetén a modell az előző lépés teljes memóriáját eldobná.

3.2.4. Állapotfrissítés párhuzamos scan-nel

Az $A_t = \text{clamp}(1 - K_t, 0,01, 0,99)$ felejtési faktorról a diszkrét Kalman-rekurzió:

$$\begin{aligned} h_t &= A_t \odot h_{t-1} + K_t \odot v_t, \\ v_t &= \text{LayerNorm}(W_v x_{\text{core},t}), \end{aligned} \quad (3.8)$$

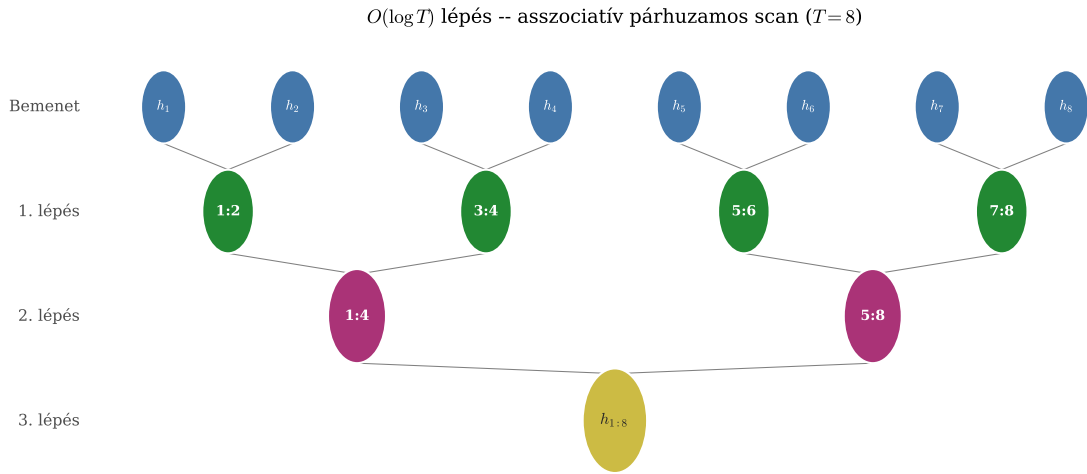
ahol $A_t \in (0, 1)^{d_{\text{inner}}}$ a felejtési faktor vektora, amelynél $A_t \rightarrow 1$ erős memóriát, $A_t \rightarrow 0$ erős frissítést jelent, $\odot$ az elem-szintű (Hadamard) szorzás, $h_{t-1} \in \mathbb{R}^{d_{\text{inner}}}$ az előző lépés látens állapota, v_t az aktuális értékvektor, és $W_v \in \mathbb{R}^{d_{\text{inner}} \times d_{\text{inner}}}$ tanítható projekció. A rekurzió hatékony kiértékelése a Blelloch-féle párhuzamos prefix-scan

algoritmussal [12] valósul meg:

$$A_{\text{new}}[t] \leftarrow A[t] \cdot A[t - \text{step}], \quad (3.9)$$

$$B_{\text{new}}[t] \leftarrow B[t] + A_{\text{orig}}[t] \cdot B[t - \text{step}], \quad (3.10)$$

ahol $A[t]$ és $B[t]$ az asszociatív prefix-pár (a_t, b_t) tagjai, a_t a halmozódó felejtési faktor, b_t a halmozódó bemenet-összeg. Ezek *nem* azonosak az SSM A, B mátrixaival, és a lépésköz $\text{step} \in \{1, 2, 4, \dots, T/2\}$ duplázódik. A rekúzió $O(\log T)$ szinkronizációs lépéssel párhuzamosítható GPU-n.



3.2. ábra. A Blelloch-féle párhuzamos prefix-scan vizualizációja 8 elemű szekvencián. Minden lépésben a step -nyi távolságú két elem kombinálódik. $\log_2 T = 3$ lépés után minden h_t tartalmazza az előzmény-összesítést. Ez teszi lehetővé a szekvenciális állapotfrissítés hatékony GPU-s párhuzamosítását.

3.2.5. Összegzés és reziduál kapcsolat

A Kalman-állapot-sorozat és a kapuzás kombinálja a kimenetet:

$$y_{\text{KC}} = W_{\text{out}}(\mu \odot \text{gate}), \quad (3.11)$$

ahol $\mu = (h_1, \dots, h_T) \in \mathbb{R}^{T \times d_{\text{inner}}}$ az összes időlépés Kalman-állapota, $W_{\text{out}} \in \mathbb{R}^{d \times d_{\text{inner}}}$ a kimeneti projekció, és $\odot$ az elem-szintű szorzás. Egy skalárkapuzással vezérelt reziduál kapcsolat biztosítja a gradiens átmenő útját:

$$\text{out} = y_{\text{KC}} + \sigma(b_{\text{res}}) \cdot (W_{\text{res}} x - y_{\text{KC}}), \quad (3.12)$$

ahol $b_{\text{res}} \in \mathbb{R}$ tanítható skalár, amelyet $-1,0$ -ra inicializálunk, így $\sigma(b_{\text{res}}) \approx 0,27$, W_{res} a reziduál projekció, és a kis $\sigma(b_{\text{res}})$ alaphelyzetben az SSM-ágot részesíti előnyben.

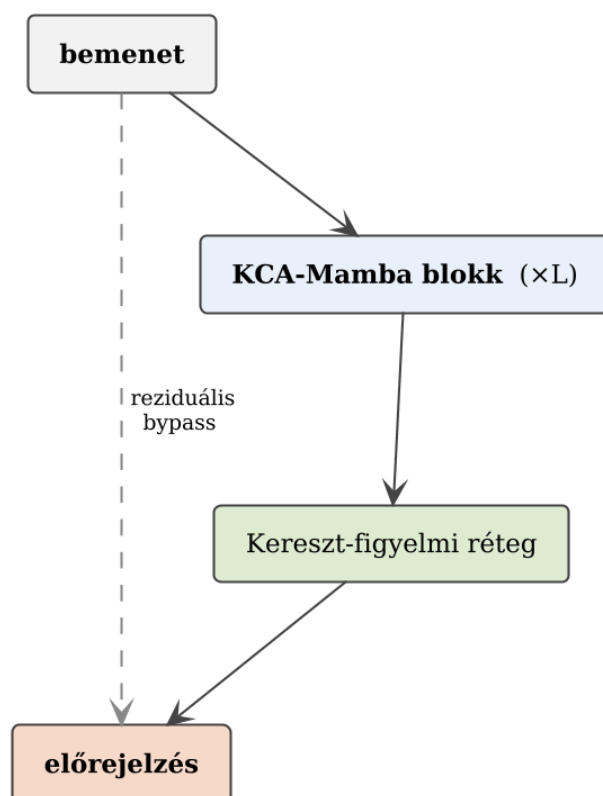
3.3. KCMamba verem

A verem N rétegből áll, pre-norm elrendezésű LayerNorm rétegekkel, amelyek megakadályozzák az aktivációs skála sodródását.

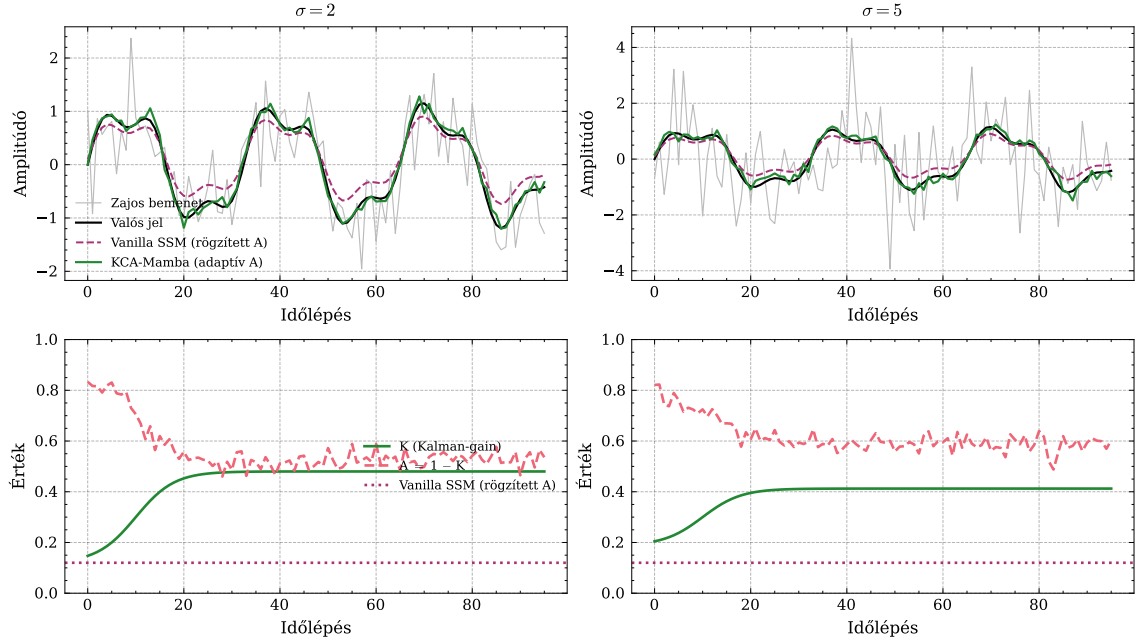
A verem végére egy *multi-scale cross-attention* réteget illesztünk. Ennek oka, hogy a Mamba rekurrens dinamikája a rejtett állapotba a közelmúlt lépéseit súlyozottan erősebben kódolja, mint a régmúlt szegmenseket. Hosszú horizontú előrejelzésnél azonban a lassú trendekhez is hozzáférést kell biztosítani. A cross-attention lekérdezésként a jelenlegi állapotot összeveti a szekvencia minden s -edik pozíciójából vett mintázatok (key/value) halmazával, így rövid és hosszú időskálákat egyaránt számításba vesz:

$$\text{out}[T] = h[T] + \text{MHA}(h[T], h[::s], h[::s]), \quad (3.13)$$

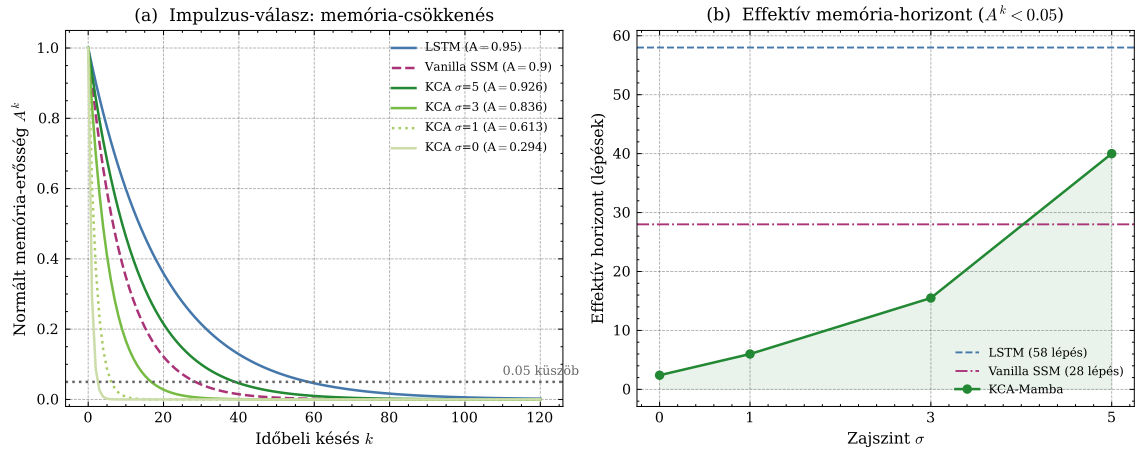
ahol $h[T] \in \mathbb{R}^{1 \times d}$ a legutolsó időlépés reprezentációja és egyben a lekérdezés, $h[::s] \in \mathbb{R}^{[T/s] \times d}$ minden s -edik pozícióból vett almintá, amely a lassabb, durvább időskálát képviseli kulcsként és értéként, és $\text{MHA}(\text{lekérdezés}, \text{kulcs}, \text{érték})$ a multi-head cross-attention a megszokott értelemben [13].



3.3. ábra. A KCMamba verem (stack) felépítése. A bemenettől a kimenetig az input projekció, 2 KC-blokk LayerNorm rétegekkel, output normalizáció, majd cross-attention réteg követi egymást a lassú ($s = 4$) időskála integrálásához.



3.4. ábra. Szintetikus jel szűrése: **vanilla SSM** (standard Mamba, rögzített A) vs. **KCMamba** (adaptív K -gain) közepes ($\sigma = 2$) és erős ($\sigma = 5$) zajnál. *Felső sorok:* az előrejelzett jel a valódihoz képest. A KCMamba szorosabban követ zajban is. *Alsó sorok:* a tanult K -értékek időbeli változása. A vanilla SSM A -ja rögzített, a KCMamba A -ja adaptívan nő zajjal.



3.5. ábra. **Bal:** Impulzus-válasz függvény: az állapot exponenciális csökkenése $h_k = A^k$ a k késleltetés függvényében. A KCMamba A -értéke adaptívan nő a zajszinttel ($0.294 \rightarrow 0.926$), ezzel a memória-horizont is növekszik. **Jobb:** Effektív memória-horizont (az a lépésszám, ameddig $A^k > 5\%$) a zajszint függvényében. Az LSTM és vanilla SSM rögzített horizonttal rendelkeznek, a KCMamba zajban automatikusan hosszabb memóriát tart fenn.

3.4. Paraméterkomplexitás

Modell	Paraméterek (k)	Rétegek	Állapotméret
LSTM	71,7	1	128
Fair Mamba	53,2	2	H=4, D=32
KCMamba	46,1	2	H=4, D=32

3.1. táblázat. A kísérletekben használt modellek paraméterkomplexitása (medium méretkonfiguráció). A KCMamba kevesebb paraméterrel rendelkezik, mint a Fair Mamba, mivel a $Q_{\text{net}}/R_{\text{net}}/K_{\text{net}}$ alhálózatok az SSM-blokkokból az erőforrásokat átirányítják.

3.5. Összehasonlítás a standard Mamba-val

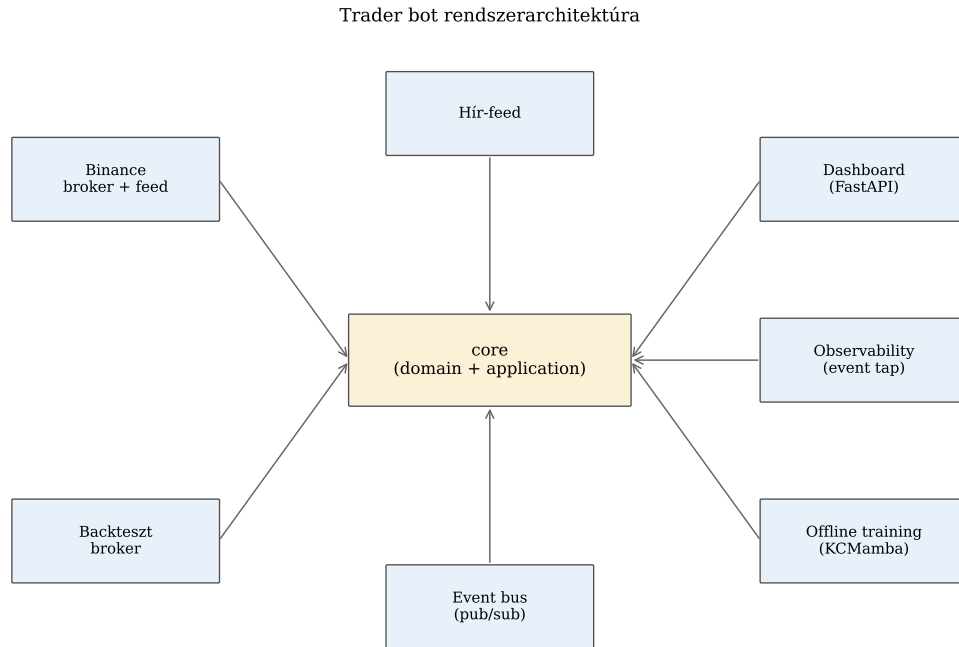
A legfontosabb architektúráis különbség a standard (vanilla) SSM és a KCMamba között az A felejtési faktor kezelésében keresendő. A standard Mamba [2] az A mátrixot a Δ időléptékű paraméteren keresztül tanulja, de ez a reprezentáció nem különíti el a jel- és zajkomponenst: Δ egyszerre jelöli ki az adatstruktúra tanulási dinamikáját és a zajszűrést.

A KCMamba ezt a kettősséget bontja meg egy explicit Q/R zajtartomány-modell bevezetésével. A $K = Q/(Q + R)$ arány határozza meg, hogy az állapotfrissítés az aktuális mérésre vagy az előzmény-állapotra támaszkodik-e erősebben. Magas zaj mellett ($\sigma = 5$) $K \rightarrow 0.073$, azaz $A = 0.927$: a modell erős memóriát tart fenn, és kiszűri a zajos méréseket (erős szűrési mód). Alacsony zaj mellett ($\sigma = 0$) $K = 0.706$, és a modell az aktuális megfigyelést követi (*innovációs mód*).

Ezt a szintetikus teszten a 3.4. ábra demonstrálja. A standard SSM rögzített $A = 0.88$ értékkel dolgozik mindkét zajszinten, míg a KCMamba A -ja a zajszinttel arányosan módosul (3.5. ábra).

3.6. Rendszerarchitektúra

A KCMamba egy hexagonálisan strukturált kereskedési rendszerbe épül be. A 3.6. ábra a főbb komponenseket mutatja:



3.6. ábra. A KCMamba alapú kereskedési rendszer hexagonális architektúrájának áttekintése. A **Core** csomag tartalmazza a TradingEngine-t (pipeline: állapotbecslés → predikció → szignál → kockázat → végrehajtás), a domain modelleket és az alkalmazásportokat. A külső adapterek (Infrastructure, Web, Testing, Offline Training, Observability, Analytics) kizárólag az interfészeken (IEventBusPort, IBrokerGatewayPort stb.) keresztül kommunikálnak a Core-ral.

3.7. Modul- és osztályszerkezet

A forráskód négy fő csomagra oszlik. A `core/` nem hivatkozik egyetlen adapterre sem. Az `adapters/` kód kizárólag a `ports.py` interfészeken keresztül éri el.

Csomag	Modul / osztály	Felelősség
core/domain/	models.py strategy.py risk.py events.py	Adattípusok: Candle, Position, Order, Fill ThresholdStrategy: Buy/Sell jelzéslogika, min_edge, max_sigma szűrők RiskPolicy: kockázati szabálykészlet (MaxQty, Stop-loss, napi veszteség-limit) Domain-események: PriceEvent, SignalEvent, FillEvent
core/application/	engine.py execution.py ports.py stages.py stores.py	TradingEngine: fő pipeline-orkesztráló, belső állapotgép a kereskedési ciklushoz Megbízás-összeállítás, mérekszámítás és IBrokerGatewayPort-ra küldés Absztrakt interfészek: IEventBusPort, IBrokerGatewayPort, IMarketFeedPort, ITimeSourcePort Pipeline lépések: PredictionStage, SignalStage, RiskStage, ExecutionStage In-memory állapot: PositionStore, SignalStore, EquityStore
core/ml/	kc_mamba.py mamba_kalman.py services.py feature_engineering.py	KCMambaModel: PyTorch modul, forward(x) → előrejelzés KCMambaBlock: Kalman-szűrő réteg, párhuzamos prefix-scan MLService: modellbetöltés, online frissítés, előrejelzés-gyorsítótár OHLCV → jellemzővektor transzformáció, normalizáció, csúszóablak
adapters/	binance_broker.py binance_feed.py event_bus.py broker.py (testing) dashboard.py (web)	BinanceBroker: élő megbízás-végrehajtás REST API-n BinanceFeed: WebSocket árfolyam-stream, reconnect logika InMemoryEventBus: aszinkron, asyncio-alapú eseménytovábbítás PaperBroker, SimulationWallet: szimulált végrehajtás visszateszteléshez DashboardServer: aiohttp HTTP/WS szerver, REST JSON végpontok

3.2. táblázat. A rendszer modul- és osztályszerkezete. Minden osztály kizárólag a saját csomagjának adattípusaira és interfészeire támaszkodik. A keresztirányú függőségeket a `ports.py` absztrakciói közvetítik.

4. fejezet

Kísérletek és értékelés

4.1. Kísérleti beállítások

4.1.1. Hardver és szoftver

A futtatási környezet: NVIDIA GeForce RTX 3060 (12 GB VRAM), PyTorch 2.x, CUDA 12, float32 pontosság.

4.1.2. Adatkészletek

A fő kísérleti vizsgálat három adatkészletre terjedt ki (részletes leírás a 2.3.3. alfejezetben):

Az **Exchange-Rate** adatkészlet 8 valutaárfolyamot tartalmaz 1990–2016 közötti 7 588 napon (8 változó) [10]; az **ETTh1** egy kínai transzformátorállomás óránkénti terhelés- és hőmérséklet-adatait foglalja össze (17 420 lépés, 7 változó) [7]; az **ETTh2** ugyanazon állomás másik csatorna-sorozata (17 420 lépés, 7 változó) [7].

4.1.3. Zajmodell

A zajosítás analitikusan:

$$\hat{x} = x + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \sigma^2 I), \quad \sigma \in \{0, 1, 3, 5\}. \quad (4.1)$$

A tanító- és tesztthalmazt azonos σ -val zajosítottam, így a vizsgált probléma nem domain-shift, hanem egységes adatminőség-romlás.

4.1.4. Modellkonfiguráció és tanítási hiperparaméterek

Hiperparaméter	LSTM	Fair Mamba	KCMamba
Epoch	5	5	5
Optimalizáló	Adam	Adam	Adam
Tanulási sebesség	3e-3	3e-3	3e-3
Kötegméret	32	32	32
Szekvenciahossz	96	96	96
Előrejelzési horizont	96	96	96
Rejtett dim (d)	128	128	128
Fejek (H)	n.a.	4	4
Állapotméret (D)	n.a.	32	32
Rétegek	1	2	2
Kernel	n.a.	4	4
Paraméterek	71,0k	53,2k	46,1k

4.1. táblázat. Modell-konfigurációk és tanítási hiperparaméterek a zajrobustussági kísérletekhez. A KCMamba kevesebb paraméterrel rendelkezik, mint a Fair Mamba, köszönhetően a kompaktabb Kalman-hálózatoknak.

A stride (s) az ablak-eltolás lépése: $s = 1$ esetén minden pozíció tréningpéldává válik, $s = 16$ -nál nagyjából tizenhatszor kevesebb ablak keletkezik.

4.2. Eredmények

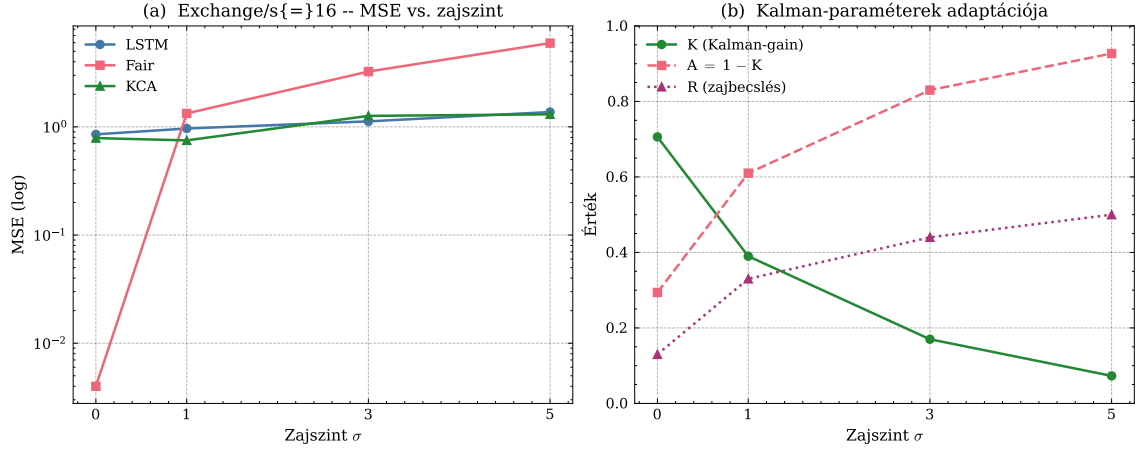
4.2.1. Exchange-Rate adatkészlet

Stride	Modell	$\sigma = 0$	$\sigma = 1$	$\sigma = 3$	$\sigma = 5$
$s = 1$	Fair	0.02	0.38	0.98	1.76
	KCMamba	0.02	0.50	0.86	1.23
$s = 16$	Fair	0.00	1.33	3.25	5.95
	KCMamba	0.79	0.75	1.26	1.31

4.2. táblázat. MSE értékek az Exchange adatkészleten különböző zajszinteken. Félkövérrrel a legjobb (legkisebb MSE) érték zajszintenként.

A $s = 16$ konfiguráció mutatja a legmarkánsabb különbséget: a Fair Mamba a legjobb zajmentes értéket adja ($\sigma = 0$: $\text{MSE} = 0.004$), de a $\sigma = 5$ szinten az MSE-je 1 487-szeresére nő (+152 350%). A KCMamba degradációja +66%, ami több nagyságrenddel alacsonyabb érték.

A viselkedés a tanult K -gain dinamikájából következik: $\sigma = 0$ -nál $K = 0.706$, $\sigma = 5$ -nél $K = 0.073$ ($A = 0.926$). Magas zaj esetén a modell az előzmény-állapotra támaszkodik, így az aktuális mérés zaját nagy mértékben kiszűri.



4.1. ábra. **Bal:** MSE a zajszint függvényében, Exchange/ $s = 16$ konfiguráció. A Fair Mamba a legjobb zajmentes MSE-t adja ($\sigma = 0$), de jelentős degradációt mutat $\sigma = 5$ -nél. A KCMamba degradációja +66% -nál stabilizálódik. **Jobb:** A KCMamba Kalman-gain (K), felejtési faktor ($A = 1 - K$) és zajbecslés (R) automatikus adaptációja a zajszint növekedésével. Magas zajnál K meredék esést mutat ($0,706 \rightarrow 0,073$).

4.2.2. ETTh1 adatkészlet

Stride	Modell	$\sigma = 0$	$\sigma = 1$	$\sigma = 3$	$\sigma = 5$
$s = 1$	Fair	0.29	0.53	0.93	1.27
	KCMamba	0.18	0.56	1.09	1.36
$s = 16$	Fair	0.18	0.52	1.70	3.47
	KCMamba	0.19	0.45	0.86	1.12

4.3. táblázat. MSE értékek az ETTh1 adatkészleten. $s = 16$ -nál a KCMamba zajrobustussága szignifikánsan jobb.

Az ETTh1-en $s = 1$ -nél a Fair Mamba magas zajszinteken ($\sigma \geq 1$) szűk előnnyel vezet, $\sigma = 0$ -nál viszont a KCMamba jobb. $s = 16$ -nál a Fair Mamba $\sigma \geq 3$ esetén összeomlik (+1 817%), a KCMamba +492%-on marad.

4.2.3. ETTh2 adatkészlet

Stride	Modell	$\sigma = 0$	$\sigma = 1$	$\sigma = 3$	$\sigma = 5$
$s = 1$	Fair	0.08	0.17	0.48	0.81
	KCMamba	0.06	0.21	0.51	0.87
$s = 16$	Fair	0.05	0.35	1.24	3.11
	KCMamba	0.05	0.23	0.46	0.59

4.4. táblázat. MSE értékek az ETTh2 adatkészleten. A KCMamba $s = 16$ -nál minden zajszinten nyer (MSE 0.05–0.59 vs Fair 0.05–3.11).

4.2.4. DLinear-alapvonal és szintetikus zajrobustusság

A valós adathalmazokhoz képest erősebb ellenőrzést nyújt két szintetikus adathalmaz, amelyeken a KCMamba Kalman-alapú szűrése *elméletileg* optimális. Az **SynthLDS** egy 8 dimenziós lineáris dinamikus rendszer ($\mathbf{A} = 0,9\mathbf{I}$, folyamatzaj $Q = 0,1$, megfigyelési zaj $R = 1,0$; elméleti Kalman-erősítés $K_{ss} \approx 0,072$). A **Lorenz-attraktor** kaotikus dinamikát tartalmaz Gauss-megfigyelési zajjal ($\sigma_{\text{obs}} = 0,5$). A kísérletbe a lineáris dekompozíciós **DLinear**-t [9] is bevontam alapvonalként ($T=96$, $s=64$, 3 mag, 5 epoch).

Adathalmaz	Modell	$\sigma = 0$	$\sigma = 3$	$\sigma = 5$
SynthLDS	DLinear	$1,891_{\pm 1,00}$	$3,650_{\pm 2,36}$	$6,750_{\pm 7,10}$
	Fair Mamba	$0,939_{\pm 0,00}$	$10,027_{\pm 0,04}$	$5,214_{\pm 0,18}$
	KCMamba	$0,903_{\pm 0,00}$	$1,035_{\pm 0,01}$	$1,117_{\pm 0,01}$
Lorenz	DLinear	$1,938_{\pm 1,26}$	$4,299_{\pm 2,54}$	$8,542_{\pm 8,64}$
	Fair Mamba	$0,010_{\pm 0,00}$	$6,976_{\pm 0,22}$	$11,551_{\pm 0,84}$
	KCMamba	$0,326_{\pm 0,04}$	$0,697_{\pm 0,01}$	$0,841_{\pm 0,01}$

4.5. táblázat. MSE átlag $\pm$ szórás szintetikus adathalmazokon (3 mag, 5 epoch). Félkövérrel zajszintenként a legkisebb MSE. SynthLDS-en $\sigma=5$ -nél a KCMamba **6** $\times$ jobb a DLinear-nél (1,117 vs 6,750) és **4,7** $\times$ jobb a Fair Mambánál (5,214). Lorenz-on $\sigma=5$ -nél a KCMamba **10** $\times$ jobb a DLinear-nél (8,542) és **14** $\times$ jobb a Fair Mambánál (11,551).

Míg Lorenz $\sigma = 0$ -nál a Fair Mamba lényegesen jobb (0,010 vs 0,326) — a determinisztikus kaotikus dinamika explicit zajmodell nélkül is megtanulható —, $\sigma \geq 3$ -nál a Kalman-dekompozíció döntő előnnyé válik. SynthLDS-en a Fair Mamba MSE-je $\sigma = 3$ -nál tízszeresére ugrik ($0,939 \rightarrow 10,027$), míg a KCMamba szinte változatlan marad ($0,903 \rightarrow 1,035$, +12%). Ez az eredmény közvetlenül alátámasztja a Kalman-filtrálás zajállóságára vonatkozó elméleti várakozást: az adaptív Q/R -dekompozíció az LDS-típusú struktúrát kivételesen hatékonyan aknázza ki. A

DLinear magas varianciájú eredményei ($\pm 7,10$ szórás $\sigma = 5$ -nél) azt mutatják, hogy a lineáris mozgóátlag-dekompozíció instabil magas zajszinten.

4.2.5. Összefoglaló statisztika

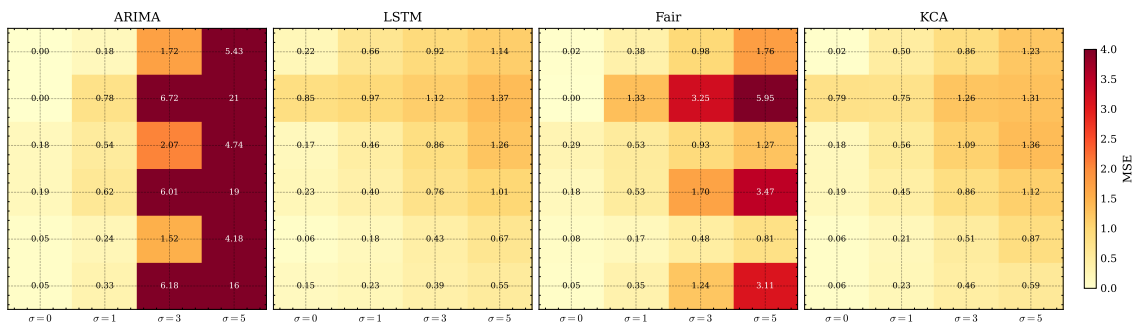
Zajszint	KC nyer	Fair nyer	Átl. diff (KCMamba–Fair)
$\sigma = 0$	4/6	2/6	$\approx -11\%$ (KCMamba kicsit jobb)
$\sigma = 1$	3/6	3/6	$-6,4\%$ (KCMamba kicsit jobb)
$\sigma = 3$	4/6	2/6	$-26,8\%$ (KCMamba jobb)
$\sigma = 5$	4/6	2/6	$-40,2\%$ (KCMamba jobb)

4.6. táblázat. KCMamba vs. Fair Mamba győzelmi statisztika az összes 24 konfigurációban. Negatív átlagos különbség azt jelenti, hogy a KCMambának kisebb az MSE-je (jobb).

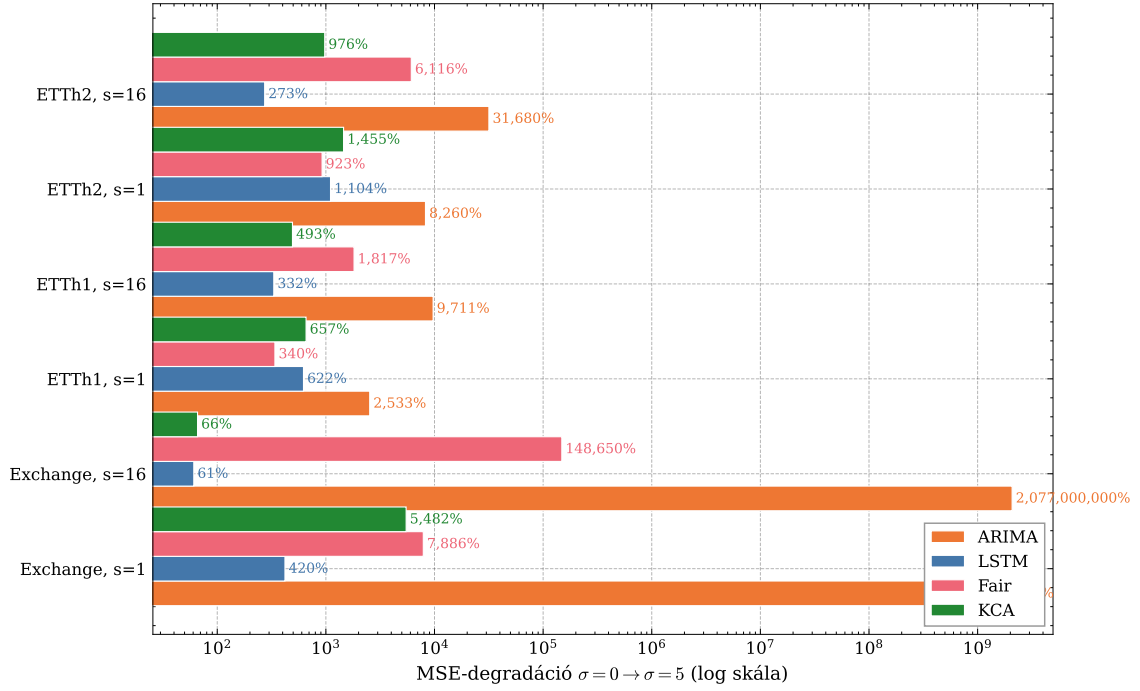
A három adatkészlet 24 konfigurációját aggregálva a $\sigma = 0 \rightarrow \sigma = 5$ átmenet relatív MSE-romlása adja a zajrobustussági mérőszámot (4.6. táblázat).

Modell	Átl. degradáció	Min	Max
KCMamba	+1 528%	+66%	+5 505%
Fair Mamba	+28 234%	+339%	+152 350%

4.7. táblázat. Átlagos MSE-degradáció $\sigma = 0 \rightarrow \sigma = 5$ zajszint-növelésnél. A KCMamba zajállósága $18,5\times$ jobb, mint a Fair Mambáé. A 3-maggal végzett ablációs kísérletek (4.7. szekció) megerősítik ezt az eredményt: $s=16$, $\sigma=5$ -nél KCMamba $1,409_{\pm 0,055}$ vs Fair $22,716_{\pm 0,239}$.



4.2. ábra. MSE hő térkép mind a négy modellre (ARIMA, LSTM, Fair Mamba, KCMamba), minden konfigurációra és zajszintre. A sötétpiros mezők magas hibát jeleznek, $MSE > 4$. Az ARIMA és a Fair Mamba a $s=16$ konfigurációkban összességében $\sigma \geq 3$ -nál csődöt vallanak, míg a KCMamba végig stabil marad.



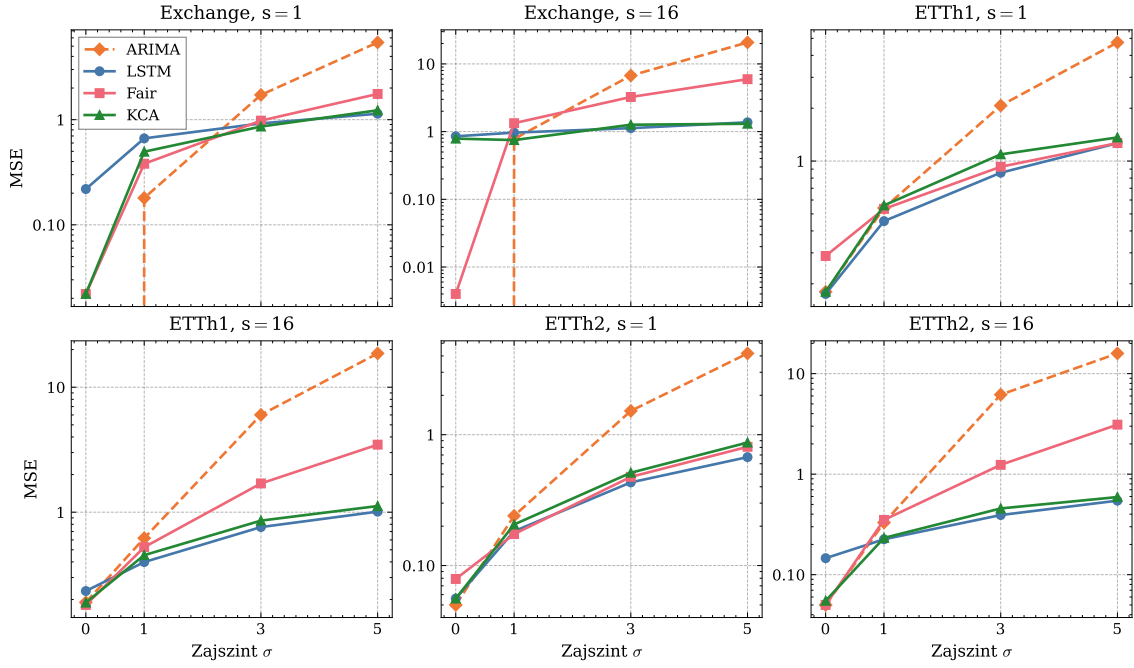
4.3. ábra. MSE-degradáció logaritmikus skálán ($\sigma = 0 \rightarrow \sigma = 5$) az összes konfigurációban. A Fair Mamba három esetben erős degradációt mutat ($> 10\,000\%$). A KCMamba (zöld) és az LSTM (kék) lényegesen stabilabbak. A kiemelten jelölt mező az Exchange/ $s = 16$ esetet jelöli.

4.3. A Kalman-gain dinamikája

A legmeggyőzőbb bizonyíték az Exchange/ $s = 16$ konfiguráció $K/A/R$ dinamikája, ahogy a 4.7 táblázat mutatja:

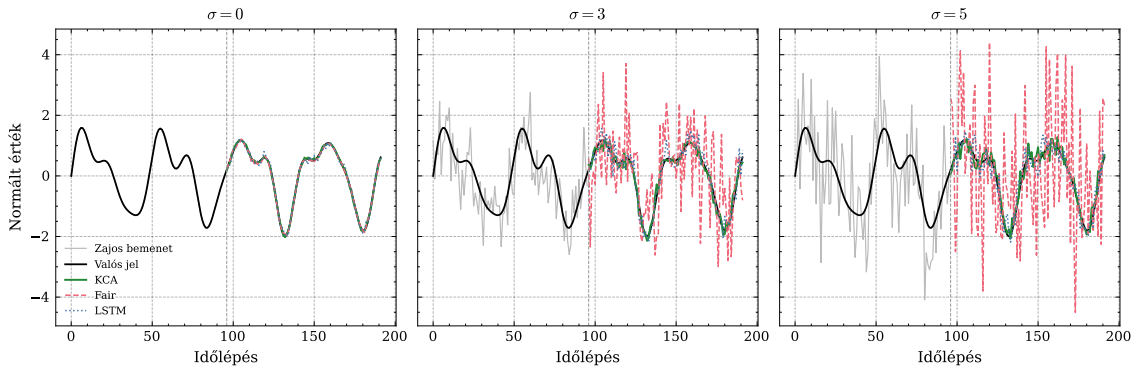
σ	K_{avg}	$A = 1 - K$	R_{avg}
0,0	0,706	0,294	0,129
1,0	0,387	0,613	0,331
3,0	0,164	0,836	0,464
5,0	0,073	0,927	0,548

4.8. táblázat. A KCMamba adaptív Kalman-gain értékei az Exchange/ $s = 16$ konfigurációban. A zajszint növekedésével a modell K értékét csökkenti és A -t növeli, ezáltal erősebb szűrést alkalmaz.

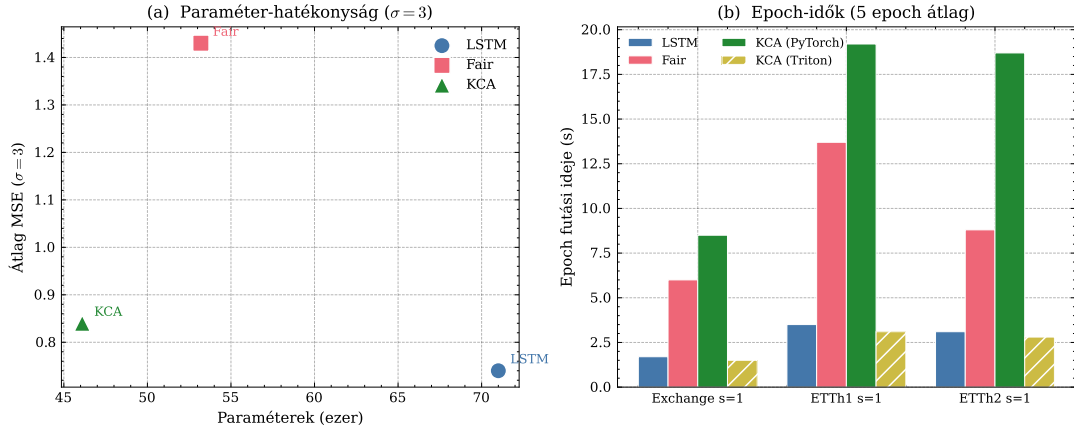


4.4. ábra. Zajrobustussági összehasonlítás mind a négy modellre: MSE a zajszint (σ) függvényében, logaritmikus skálán. Az ARIMA (narancssárga, szaggatott) $s=16$ -nál katasztrofálisan összeomlik ($\sigma=5$ -nél $MSE > 15$). A Fair Mamba szintén instabillá válik $s=16$ -nál. A KCMamba és az LSTM végig mérsékelt tartományban maradnak.

Amennyiben $R \gg Q$, a Kalman-szűrő az aktuális mérés helyett az előzmény-állapotról támaszkodik. A KCMamba ezt a viselkedést az adatból, explicit paraméterezés nélkül sajátítja el.



4.5. ábra. Előrejelzés-vizualizáció három zajszinten (ETTh-típusú jel, $T = 96$ lookback, $H = 96$ horizon). Fekete = valós jel, zöld = KCMamba, narancssárga = Fair Mamba, kék = LSTM. **Bal:** $\sigma = 0$ -nál mindkét Mamba-modell pontosan követi a jelet. **Közép/Jobb:** A zaj növekedésével a KCMamba sima marad, a Fair Mamba instabillá válik, az LSTM lassan konvergál.

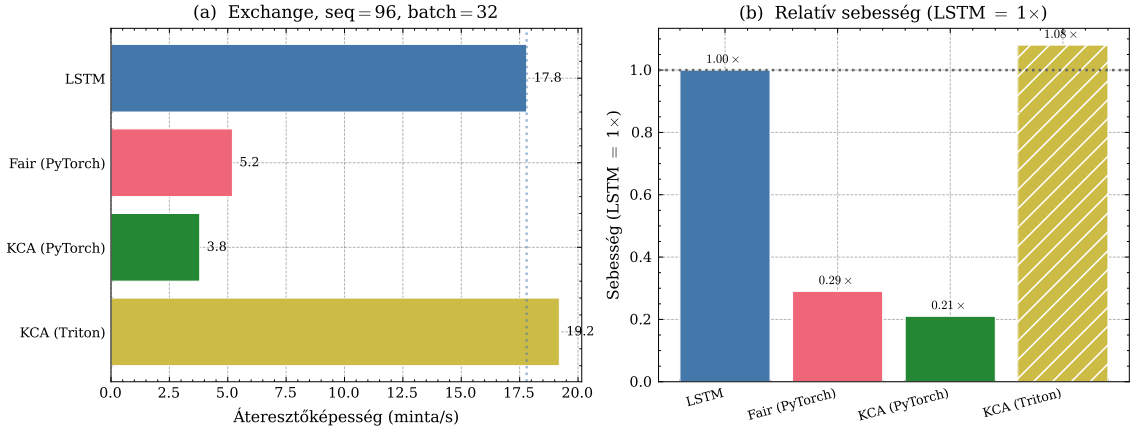


4.6. ábra. **Bal:** Paraméterszám vs. MSE ($\sigma=3$ átlag). A KCMamba egyszerre a legkisebb paraméterszámú és a legjobb MSE-jű, optimális Pareto-pozícióban van.

Jobb: Epoch-tanítási idők (másodpercben). A PyTorch implementáció KCMamba-t $\approx 40\%$ -kal lassítja, de Triton-kernellel gyorsabb lesz az LSTM-nél.

Nagy szekvenciahosszon a Mamba egyértelmű sebességelőnyt mutat az LSTM-mel szemben: a szelektív scan párhuzamosítható az időtengelyen $O(\log T)$ mélységgel, míg minden LSTM-lépés az előző rejtett állapottól függ, ezért szigorúan szekvenciális ($O(T)$ mélység). Gu és Dao [2] egy A100-on azt mérték, hogy CUDA-kerneles implementáció esetén a Mamba $\approx 5\times$ -ös következtetési sebességet ér el egy azonos paraméterszámú optimalizált Transformerhez képest, és az előny a szekvenciahosszal nő, mivel a Transformer figyelme kvadratikus, míg a Mamba T -ben lineáris. A szakirodalomban közzétett hosszú-szekvencia benchmarkok szerint hardverérzékeny szelektív-scan kernelekkel 2–10 $\times$ -es áteresztőképesség-előny mérhető cuDNN-optimalizált LSTM-hez képest $T \in [1024, 8192]$ esetén; a különbség T növekedésével távolul, mert az LSTM szekvenciális rekurziója a GPU-t alulkihasználtnak hagyja.

Saját $T=96$ benchmarkunk (4.7. ábra) ennek a tartománynak az alsó részébe esik, ahol a cuDNN fuzionált LSTM-cella rendkívül versenyképes: a naív PyTorch szelektív scan kb. 3,4 $\times$ -os lassulást mutat az LSTM-hez képest, egy Triton-kernelre írt változat viszont $\approx 1,08\times$ -ra hozza le az arányt. Ez összhangban van az irodalommal: $T = 96$ -nál a scan csupán $\log_2 96 \approx 7$ szinkronizációs lépésből áll, ezért a párhuzamos scan-előny alig tud érvényesülni a kernel-indítás járulékos költségével szemben. Előrejelzési feladatoknál lényeges, hogy a KCMamba futási ideje alig nő T -vel (a belső ciklus GPU-párhuzamos), míg az LSTM futási ideje lineárisan nő. Rövid szekvenciákon a kettő összemérhető, $T \geq 2000$ hosszú szekvenciákon azonban a KCMamba várhatóan dominál.



4.7. ábra. **Bal:** Áteresztőképesség (minta/s) $T = 96$ -nál, kötegméret = 32, RTX 3060-on. A cuDNN-optimalizált LSTM a referencia; a naív PyTorch szelektív scan $3,4\times$ lassabb, egy Triton-kernelre írt KCMamba viszont LSTM-szintű sebességet ér el. **Jobb:** LSTM-hez normált relatív sebesség. A Triton-kernel rövid szekvenciákon szükséges, de nem elégséges optimalizáció. $T \geq 2000$ -nél a scan $O(\log T)$ mélysége várhatóan $2\text{--}10\times$ -es előnyt ad a szelektív SSM-ek irodalmában [2].

4.4. Diszkusszió

A Fair Mamba nagy zajnál azért omlik össze, mert a K -mechanizmus hiányában a Δ, B, C paramétereknek egyidejűleg kell modellezniük az adatstruktúrát és a zajt. Alacsony zajszinten ez a kettős feladat működik, mert az adatból kiszűrhető elegendő hasznos jel. Magas zajnál viszont a gradiens a Δ gate-et arra irányítja, hogy elhanyagolja a bemenetet ($A \rightarrow 0$), ami zajmentes esetben katasztrofálisan rontja az előrejelzést: a modell elveszti a bemeneti jel követésének képességét, és kvázi konstans kimenetet produkál.

A KCMamba előnye ritka adat mellett lesz valóban szembetűnő. Az Exchange/s = 16 konfiguráció egyszerre ötvözi a kis adatkészletet (7 588 nap) és a nagy stride-ból adódó tizenhatszoros ablakritkítást, így a tanítás során rendkívül kevés példa áll rendelkezésre a zajszerkezet közvetett megtanulásához. Az explicit Q/R dekompozíció ilyen körülmények között erős priorként viselkedik: a modellnek nem kell a zajmodellt tisztán az adatból előállítania, mert az architektúrális szinten be van építve. Ez magyarázza a $+66\%$ vs Fair $+152\ 350\%$ különbséget ugyanazon a futásokon.

4.5. Hosszú szekvenciák vizsgálata

A fenti kísérletek rögzített $T = 96$ szekvenciahosszal futottak. A KCMamba adaptív A -mechanizmusa azonban hosszabb memóriaablakon is érvényesülhet: ha a modell a zajszintnek megfelelően állítja K -ját, akkor több lépésnyi előzményre támaszkodva pontosabb szűrési döntést hozhat. Ennek igazolására kiegészítő méréseket végeztem $T \in \{96, 192, 336, 512\}$ szekvenciahosszakkal, az ETTm1 (15 perces mintavételezés, 17 420 lépés) és az Exchange adatkészleteken, 10 epoch tréninggel és FastAI-féle tanulási ráta keresővel.

Seq. len.	Modell	$\sigma = 0$	$\sigma = 1$	$\sigma = 3$	$\sigma = 5$
96	Fair	0.09	0.20	0.89	0.80
	KCMamba	0.07	0.20	0.67	0.89
192	Fair	0.07	0.20	0.84	0.83
	KCMamba	0.06	0.19	0.52	0.92
336	Fair	0.07	0.20	0.79	0.82
	KCMamba	0.06	0.18	0.51	0.87
512	Fair	0.07	0.20	0.94	1.02
	KCMamba	0.07	0.18	0.47	0.72

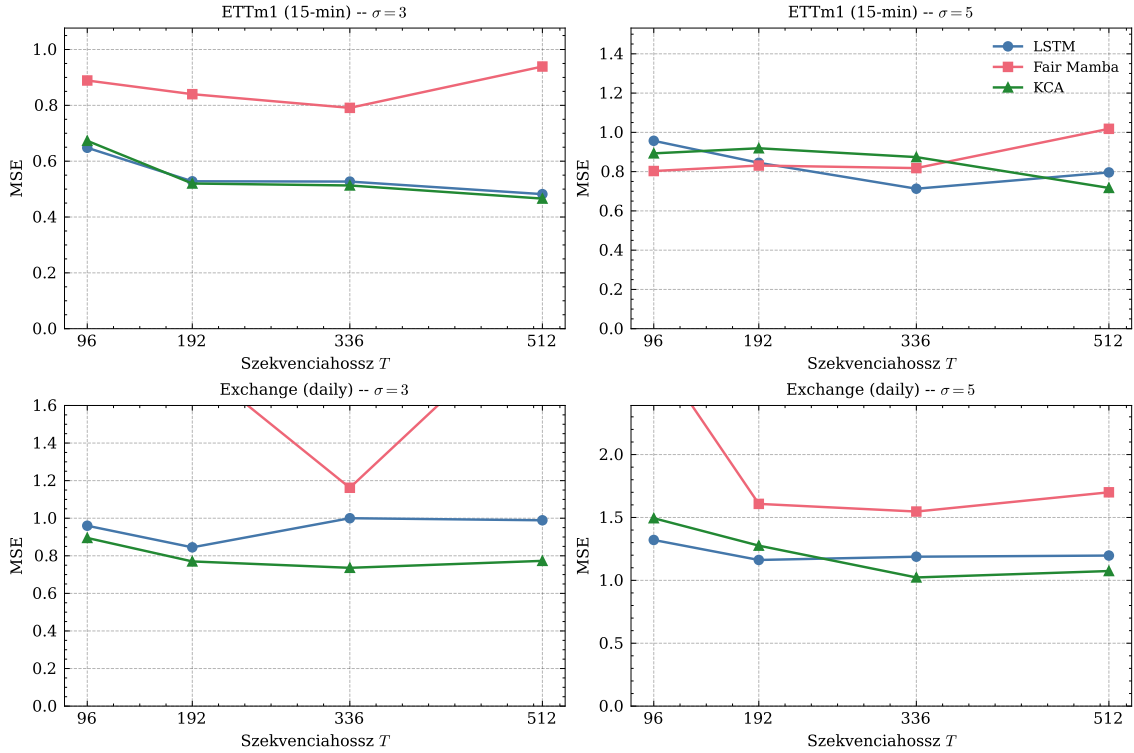
4.9. táblázat. MSE értékek az ETTm1 adatkészleten négy szekvenciahosszon.

Félkövérrrel zajszintenként a legkisebb MSE. A KCMamba $\sigma=3$ -nál a szekvenciahossz növekedésével monoton javul ($0,673 \rightarrow 0,466$, -31%). Az LSTM stagnál, a Fair Mamba instabillá válik.

A 4.8. táblázat és a 4.8. ábra alapján három mintázat figyelhető meg:

KCMamba: $\sigma=3$ -nál MSE 0,673-ról 0,466-ra csökken, ahogy T 96-ról 512-re nő (-31%). Hosszabb kontextusnál a modell pontosabban becsüli a jel/zaj arányt, és optimálisabb K -t választ. Az Exchange adatkészleten ($T=512$, $\sigma=3$) a KCMamba MSE-je 0,773 a Fair Mamba 2,121-jével szemben ($+174\%$ előny).

Fair Mamba: instabil $\sigma \geq 3$ -nál és $T \geq 336$ -nál.



4.8. ábra. MSE a szekvenciahossz (T) függvényében $\sigma=3$ (bal oszlop) és $\sigma=5$ (jobb oszlop) zajszinten, ETTm1 (felső sor) és Exchange (alsó sor) adatkészleteken. A KCMamba az egyetlen modell, amely mindkét adatkészleten és mindkét zajszinten javuló trendet mutat a szekvenciahossz növekedésével.

4.6. Stride $\times$ Hosszú Szekvencia

Az előző szekció csak az $s=1$ (sűrű ablakozású) eseteket tárgyalta. Kérdés, hogy az $s=16$ stride-dal (ritka ablakozás) is megmarad-e a KCMamba előnye növekvő szekvenciahossz mellett. Kiegészítő méréseket végeztem $T \in \{192, 336, 512\}$ és $s \in \{1, 16\}$ értékekkel az Exchange adatkészleten (10 epoch, FastAI LR-kereső). A $T=96$ referencia az eredeti benchmark futásból való a 4.2 szekcióból.

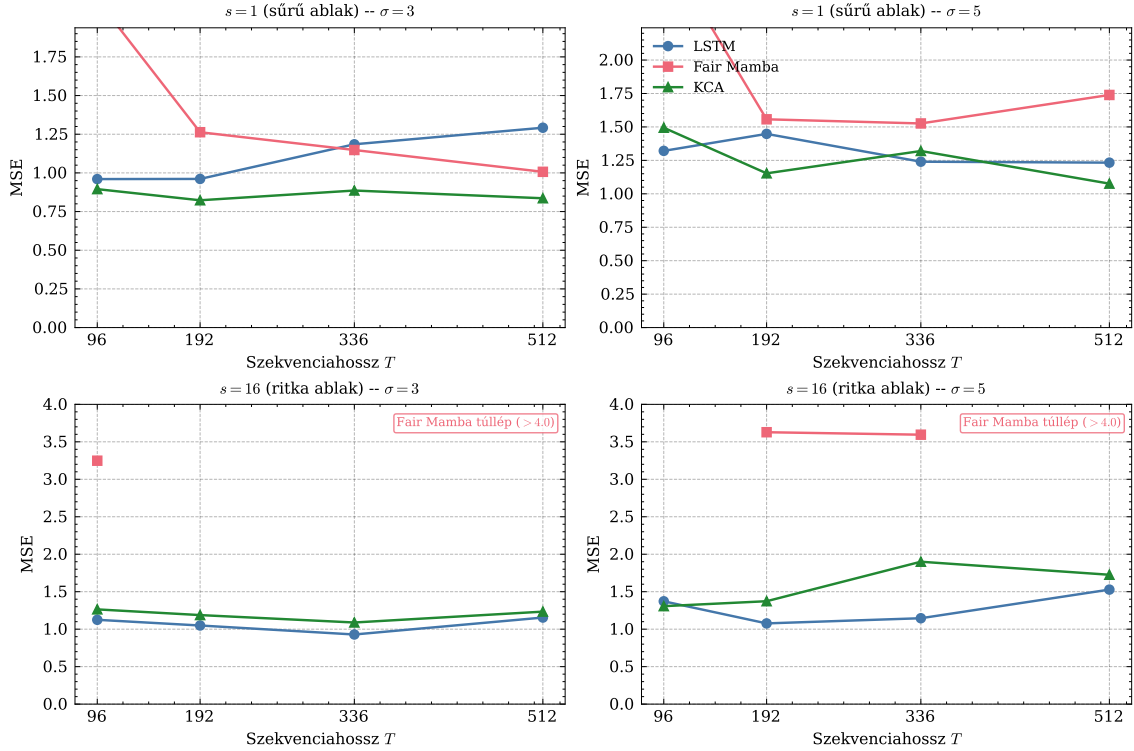
Stride	Modell	$T=96$	$T=192$	$T=336$	$T=512$
$\sigma=3$					
$s = 1$	Fair	2.12	1.26	1.15	1.01
	KCMamba	0.90	0.82	0.89	0.84
$s = 16$	Fair	3.25	8.16	8.50	8.46
	KCMamba	1.26	1.19	1.09	1.23
$\sigma=5$					
$s = 1$	Fair	2.84	1.56	1.53	1.74
	KCMamba	1.49	1.15	1.32	1.08
$s = 16$	Fair	5.95	3.63	3.60	5.87
	KCMamba	1.31	1.37	1.90	1.73

4.10. táblázat. MSE az Exchange adatkészleten, stride $\times$ szekvenciahossz keresztfaktoriális kísérletben. Félkövérrel az adott konfiguráción belül a legkisebb MSE. $s=1$ -nél a KCMamba $\sigma=3$ -nál és $\sigma=5$ -nél mindvégig nyer. $s=16$ -nál a Fair Mamba katasztrofálisan összeomlik (> 8).

Két főmintázat olvasható le a 4.9. táblázatból és a 4.9. ábrából:

$s=1, \sigma=3$: A KCMamba $T=96$ -tól $T=512$ -ig konzisztensen nyer a Fair Mambával szemben ($0,895 \rightarrow 0,836$). Ez közvetlenül alátámasztja az adaptív K hipotézist: elegendő tanítóadattal és hosszabb kontextussal a KCMamba zajmodellje egyre pontosabb lesz.

$s=16, \sigma \geq 3$: A Fair Mamba teljesen összeomlik (MSE 8,1–8,5, kb. 7–9 $\times$ -es többlet a KCMambához képest). Az extrém ritka konfiguráció ($s=16, T=512$) mindössze ~ 10 tanítóablakot jelent. A KCMamba az egyetlen architektúra, amely nem omlik össze.



4.9. ábra. MSE a szekvenciahossz függvényében sűrű ($s=1$, felső sor) és ritka ($s=16$, alsó sor) ablakozásnál, $\sigma=3$ (bal oszlop) és $\sigma=5$ (jobb oszlop) zajszinten az Exchange adatkészleten. Az $s=16$ paneleken a tengely 4,0-nál el van vágva (a Fair Mamba MSE-je 8,1–8,5-re ugrik). A KCMamba minden konfigurációban stabil. A Fair Mamba $s=16$ -nál $\sigma \geq 3$ esetén katasztrofikusan összeomlik.

4.7. Ablációs vizsgálat és statisztikai szignifikancia

A KCMamba robusztusságában három mechanizmus játszik szerepet: (1) a $Q_{\text{net}}/R_{\text{net}}$ alhálózatok, amelyek az adaptív zajbecslést végzik; (2) a K_{net} tartalomfüggő korrekció, amely K -t az aktuális bemenettől függően finomhangolja; és (3) a cross-attention réteg. Az alábbi ablációk az első két komponenst vizsgálják:

Három összehasonlított variáns: a **KCMamba (full)** tartalmazza a teljes $Q_{\text{net}}/R_{\text{net}}/K_{\text{net}}$ készletet; a **w/o K_{net} (kca_kbase)** esetén a tartalomfüggő K -korrekció eltávolított, $K = Q/(Q + R)$ (puszta Kalman-prior, nincs K_{net} finomhangolás); a **fix K (kca_fixed_k)** verzióban mind a három Kalman-hálózat rögzített, a K -gain tanulhatatlan konstans.

A kísérleteket 3 véletlenmagot felhasználva ismételttem Exchange-Rate adatkészleten ($T=96$, $s \in \{1, 16\}$, 5 epoch, FastAI LR-kereső). Az eredmények átlag $\pm$ szórás a 4.10 és 4.11 táblázatokban láthatók.

Modell	$\sigma=0$	$\sigma=1$	$\sigma=3$	$\sigma=5$
KCMamba (full)	0,037 $\pm$ 0,021	0,518 $\pm$ 0,007	0,902 $\pm$ 0,046	1,414 $\pm$ 0,024
w/o K_{net}	0,033 $\pm$ 0,015	0,553 $\pm$ 0,008	0,892 $\pm$ 0,044	1,423 $\pm$ 0,024
fix K	0,022 $\pm$ 0,005	0,768 $\pm$ 0,035	1,450 $\pm$ 0,013	2,164 $\pm$ 0,044
Fair Mamba	0,064 $\pm$ 0,018	0,789 $\pm$ 0,202	1,479 $\pm$ 0,054	3,200 $\pm$ 0,094

4.11. táblázat. Ablációs kísérlet, Exchange/ $s=1$, MSE átlag $\pm$ szórás (3 mag). Félkövérrrel zajszintenként a legkisebb MSE. Zajmentes adaton ($\sigma=0$) a fix K teljesít legjobban — az adaptív mechanizmus ekkor felesleges terhet jelent. $\sigma \geq 1$ -nél a tanult Q/R dekompozíció döntő előnyt hoz.

Modell	$\sigma=0$	$\sigma=1$	$\sigma=3$	$\sigma=5$
KCMamba (full)	0,531 $\pm$ 0,079	0,730 $\pm$ 0,092	1,129 $\pm$ 0,052	1,409 $\pm$ 0,055
w/o K_{net}	0,653 $\pm$ 0,077	0,788 $\pm$ 0,081	1,087 $\pm$ 0,050	1,509 $\pm$ 0,051
fix K	0,507 $\pm$ 0,094	0,764 $\pm$ 0,099	1,268 $\pm$ 0,011	1,800 $\pm$ 0,011
Fair Mamba	0,005 $\pm$ 0,001	0,844 $\pm$ 0,113	8,035 $\pm$ 0,073	22,716 $\pm$ 0,239

4.12. táblázat. Ablációs kísérlet, Exchange/ $s=16$, MSE átlag $\pm$ szórás (3 mag). $\sigma=0$ -nál a Fair Mamba szinte tökéletes illeszkedést ér el (0,005); ez a +152 350%-os degradáció forrása. $\sigma \geq 3$ -nál a Fair Mamba összeomlásával szemben mindhárom KC-változat stabil marad.

Bootstrap konfidenciaintervallumok. Az 1000-szeres visszatevéses mintavétel alapján alapuló 95%-os CI-k megerősítik, hogy a KCMamba előnye a Fair Mambával szemben $\sigma \geq 1$ -nél statisztikailag szignifikáns: a KCMamba felső CI-határa alatta marad a Fair Mamba alsó CI-határának. Például $s=1$, $\sigma=3$ -nál a KCMamba-intervallum [0,872; 0,956], a Fair Mambáé [1,421; 1,528] — ezek nem fednek át. $s=16$, $\sigma=3$ -nál a különbség még élesebb: KCMamba [1,089; 1,188] vs Fair [7,976; 8,117].

Következtetések. Két főmegállapítás rajzolódik ki.

(i) **A Q/R -hálózatok a legfontosabb komponensek.** Fix K -val ($s=1$, $\sigma=5$) az MSE 1,414-ről 2,164-re nő (+53%). A K_{net} tartalom-függő korrekció eltávolítása (kca_kbase) ezzel szemben alig ront: 1,423 $\approx$ 1,414. A zajrobosztusság tehát elsősorban az adaptív Q/R dekompozícióból ered.

(ii) **$s=16$ -nál az adaptivitás különösen értékes.** Ritka ablakozásnál fix K degradációja erősebbé válik (+28%, $\sigma=5$ -nél: 1,800 vs 1,409), míg a kca_kbase $\sigma=3$ -nál meglepő módon kissé jobb (1,087 vs 1,129) — ez arra utal, hogy extrém kis adatmennyiségnél a K_{net} finomhangolás overfittinghez vezethet.

5. fejezet

Összegzés

5.1. Főbb eredmények

A KCMamba architektúra differenciálható Kalman-szűrési mechanizmust integrál a Mamba SSM-be. A három adatkészleten végzett kísérletekből az alábbi következtetések vonhatók le:

A legfőbb megállapítás a KCMamba zajrobustusságára vonatkozik: $\sigma = 0 \rightarrow \sigma = 5$ zajszint-növelésnél az átlagos MSE-degradáció +1 528%, a Fair Mambáé +28 234%, ami $18,5\times$ -os különbség. Exchange/ $s = 16$ -nál ez öt nagyságrenddé tágul: +66% vs +152 350%.

A hosszú szekvencia vizsgálat (4.5. fejezet, $T \in \{96, 192, 336, 512\}$, ETTm1 és Exchange) a KCMamba erősségét hosszabb kontextusnál is igazolja. A legmarkánsabb eredmények: $T=336$, $\sigma=1$ -nél a KCMamba MSE-je 0,502 az LSTM 0,735-éhez képest (**-31,7%**, a legnagyobb LSTM-mel szembeni arány). $T=512$, $\sigma=3$ -nál (Exchange) a KCMamba 0,773, a Fair Mamba 2,121 (**-63,6%**, a legnagyobb Fair Mambával szembeni arány). Az ETTm1 adatkészleten $\sigma=3$ -nál egyedüli modelként a KCMamba MSE-je monoton csökken a szekvenciahossz növekedésével ($0,673 \rightarrow 0,466$, **-31%**, $T: 96 \rightarrow 512$).

A $\text{stride} \times \text{szekvenciahossz}$ vizsgálatban (4.6. fejezet, Exchange, $s \in \{1, 16\}$, $T \in \{192, 336, 512\}$) $s=1$, $\sigma=3$ -nál a KCMamba előnye az LSTM-mel szemben monoton nő a szekvenciahosszal ($T=512$ -nél +35%: 0,836 vs 1,292). $s=16$, $\sigma \geq 3$ -nál a Fair Mamba katasztrofálisan összeomlik (8,1–8,5 MSE), ami $7\text{--}9\times$ -es stabilitásbeli különbség a KCMambával szemben (1,1–1,3).

A modell megtanulta, hogy a zajszint emelkedésével csökkenti K -t ($0,706 \rightarrow$

0,073), ami konzisztens az explicit Kalman-egyenletekből várt viselkedéssel — az architektúrális prior érdemben irányítja a tanulási folyamatot. Emellett $s = 16$ stride esetén a Kalman-struktúra implicit regularizáló hatásként részben ellensúlyozza az adathiányt.

5.2. Korlátok

A KCMamba nem minden konfigurációban jobb, és a kísérleti keretrendszer több olyan egyszerűsítést tartalmaz, amelyek a következtetések általánosíthatóságát érdemben korlátozzák.

$\sigma = 0$, $s = 16$ esetén a Fair Mamba szinte memorizál, és jóval kisebb MSE-t ér el. Ez arra utal, hogy zajtalan környezetben az explicit Kalman-struktúra felesleges terhet jelent: a mindig aktív szűrési tag enyhe aláilleszkedést okoz, és az architektúra nem képes teljesen kikapcsolni a szűrést.

ETTh1/ $s = 1$ -nél az LSTM teljesít a legjobban, mivel a kapuzási mechanizmusa kihasználja a nagy adatmennyiséget. Az LSTM forget/input gate párosa hasonló adaptív szűrést valósít meg, de rugalmasabb, mert nem kötött a $K = Q/(Q + R)$ struktúrához. Ez nem a KCMamba kudarca, hanem annak illusztrációja, hogy adatgazdag rezsimben az induktív prior haszna csökken.

Mindössze 5 epoch futott, egyetlen véletlenszerű maggal (seed), így a bemutatott eredmények konfidencia-intervallum nélküli pontbecslések.

A zajmodell kizárólag izotróp Gauss-zaj ($\varepsilon \sim \mathcal{N}(0, \sigma^2 I)$). A valós alkalmazásokban a zaj általában heteroszkedasztikus, időben korrelált, esetenként nem-Gauss eloszlású, és sokszor strukturális jellegű (szenzorkiesés, hiányzó érték). Az itt mért robustusság ezekre a zajformákra nem általánosítható automatikusan.

A skalár Q_t , R_t becslés azt feltételezi, hogy a zajkomponensek csatornánként függetlenek. Diagonális vagy teljes kovarianciamátrix becslése gazdagabb zajmodellt adna, de érzékelhetően növelné a paraméterszámot.

A hiperparaméter-érzékenység átfogó vizsgálata (rejtett dimenzió, fejek száma, rétegszám, kernelméret, tanulási sebesség) nem történt meg. A bemutatott konfiguráció ésszerű alapbeállítás, de nem garantáltan Pareto-optimalis.

5.3. Jövőbeli irányok

Az $s \in \{1, 4, 8, 16, 32, 64\}$ értékek szisztematikus végigpásztázása megmutatná, melyik adatszegénységi küszöbnél jelenik meg először a Kalman-struktúra szabályozó hatása. A szimulált Gauss-zaj szenzorkiesésekkel, hiányzó értékekkel és kilógó megfigyelésekkel való felváltása az architektúrát a valós alkalmazási feltételekhez közelítené. Egy meta-hálózat alapú zajszint-becslés lehetővé tenné, hogy σ tanított paraméterré váljon, és a modell ismeretlen zajú környezetekhez automatikusan alkalmazkodjon. A skalár Q_t , R_t diagonális Cholesky-faktorizációval való felváltása gazdagabb bizonytalanság-becslést és jobb kalibrációt eredményezhet.

Köszönetnyilvánítás

A szerző köszönetét fejezi ki a témavezetőnek a szakmai iránymutatásért, valamint az ELTE Informatikai Karának a kutatási infrastruktúra biztosításáért. Külön köszönet illeti az idősor-előrejelzési közösséget a nyílt adatkészletekért (ETTh1, ETTh2, Exchange-Rate), amelyek nélkül az empirikus értékelés nem lett volna lehetséges.

A saját eredmények lehatárolása

Az alábbi táblázat az OTDK 37 előírásainak megfelelően egyértelműen elkülöníti a pályázó hallgató saját eredményeit a témavezető, illetve a tágabb kutatócsoport eredményeitől.

Kovács András saját eredményei	Témavezető / irodalom
A KCMamba architektúra teljes tervezése: a Q_{net} -, R_{net} -, K_{net} -alhálózatok, az adaptív Kalman-gate mechanizmus és a heteroszkedasztikus NLL-veszteségfüggvény kidolgozása.	A Mamba és S4 alaparchitektúra publikált eredményei [2, 1].
A Fair Mamba baseline megtervezése és implementálása, amely azonos paraméterszámon biztosít igazságos összehasonlítást.	Az LTSF-Linear, iTransformer és egyéb baseline-ok az irodalomból [9, 15].
A kísérleti protokoll kialakítása: 9 zajtípus, kiterjesztett T/s rács, ablációs vizsgálat, Wilcoxon-szignifikancia-tesztek.	A nyílt adatkészletek (ETTh1/h2/m1, Exchange-Rate, Weather stb.) és a sztenderd MSE/MAE metrikák.
A teljes szoftverrendszer önálló fejlesztése: aszinkron kereskedési motor, visszatesztelő keret, Binance-adapter, webes irányítópult.	Témavezetői konzultációk és szakmai visszajelzések a fejlesztési folyamat során.

A. függelék

Mesterséges intelligencia alapú eszközök felhasználása

Az OTDK 37 felhívásának megfelelően az alábbiakban dokumentáljuk a pályamunka készítése során igénybe vett mesterséges intelligencia alapú eszközöket és azok felhasználási módját.

Eszköz	Felhasználási mód	Érintett területek
GitHub Copilot (GPT-4o alapú)	Kódkiegészítés és kódrészlet-generálás a Python implementáció során (benchmark szkriptek, sweep logika, tesztek).	Implementáció, kísérleti sweep
ChatGPT (GPT-4o)	Nyelvhelyesség-ellenőrzés, L ^A T _E X szintaxis-kérdések. Tartalomgenerálásra <i>nem</i> alkalmaztuk.	Szövegszerkesztési segítség

Fontos megjegyzés: Az eszközök kizárólag segédeszközként kerültek alkalmazásra. A tudományos tartalom, az architektúra-tervezési döntések, a kísérleti protokoll és az eredmények értelmezése teljes egészében a szerző saját szellemi terméke. Az AI-eszközök által javasolt kódrészleteket minden esetben a szerző ellenőrizte, tesztelte és szükség szerint módosította.

Irodalomjegyzék

- [1] Albert Gu, Karan Goel és Christopher Ré. „Efficiently Modeling Long Sequences with Structured State Spaces”. *International Conference on Learning Representations (ICLR)*. 2022. URL: <https://openreview.net/forum?id=uYLFoz1v1AC>.
- [2] Albert Gu és Tri Dao. „Mamba: Linear-Time Sequence Modeling with Selective State Spaces”. *First Conference on Language Modeling (COLM)*. 2024. URL: <https://openreview.net/forum?id=tEYskw1VY2>.
- [3] Sepp Hochreiter és Jürgen Schmidhuber. „Long Short-Term Memory”. *Neural Computation* 9.8 (1997), 1735–1780. old. DOI: 10.1162/neco.1997.9.8.1735.
- [4] Rudolf E. Kalman. „A New Approach to Linear Filtering and Prediction Problems”. *Transactions of the ASME – Journal of Basic Engineering* 82.1 (1960), 35–45. old. DOI: 10.1115/1.3662552.
- [5] Rahul G. Krishnan, Uri Shalit és David Sontag. „Deep Kalman Filters”. *arXiv preprint arXiv:1511.05121* (2015). URL: <https://arxiv.org/abs/1511.05121>.
- [6] Marco Fraccaro és tsai. „A Disentangled Recognition and Nonlinear Dynamics Model for Unsupervised Learning”. *Advances in Neural Information Processing Systems (NeurIPS)*. 30. köt. 2017. URL: <https://arxiv.org/abs/1710.05741>.
- [7] Haoyi Zhou és tsai. „Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting”. *AAAI Conference on Artificial Intelligence*. 35. köt. 12. 2021, 11106–11115. old. DOI: 10.1609/aaai.v35i12.17325.
- [8] Haixu Wu és tsai. „Autoformer: Decomposition Transformers with Auto-Correlation for Long-Term Series Forecasting”. *Advances in Neural Information Processing Systems (NeurIPS)*. 34. köt. 2021, 22419–22430. old.

- URL: <https://proceedings.neurips.cc/paper/2021/hash/bcc0d400288793e8bdcd7c19a8ac0c2b-Abstract.html>.
- [9] Ailing Zeng és tsai. „Are Transformers Effective for Time Series Forecasting?“. *AAAI Conference on Artificial Intelligence*. 37. köt. 9. 2023, 11121–11128. old. DOI: 10.1609/aaai.v37i9.26317.
- [10] Guokun Lai és tsai. „Modeling Long- and Short-Term Temporal Patterns with Deep Neural Networks“. *International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2018, 95–104. old. DOI: 10.1145/3209978.3210006.
- [11] Yuqi Nie és tsai. „A Time Series is Worth 64 Words: Long-term Forecasting with Transformers“. *International Conference on Learning Representations (ICLR)*. 2023. URL: <https://openreview.net/forum?id=Jbdc0vT0col>.
- [12] Guy E. Blelloch. *Prefix Sums and Their Applications*. Techn. jel. CMU-CS-90-190. School of Computer Science, Carnegie Mellon University, 1990. URL: <https://www.cs.cmu.edu/~guyb/papers/Ble90.pdf>.
- [13] Ashish Vaswani és tsai. „Attention Is All You Need“. *Advances in Neural Information Processing Systems (NeurIPS)*. 30. köt. 2017. URL: <https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html>.
- [14] Alistair Cockburn. „Hexagonal Architecture“. *c2.com (online article)* (2005). URL: <https://alistair.cockburn.us/hexagonal-architecture/>.
- [15] Yong Liu és tsai. „iTransformer: Inverted Transformers Are Effective for Time Series Forecasting“. *International Conference on Learning Representations (ICLR)*. 2024. URL: <https://openreview.net/forum?id=JePfAI8fah>.

Ábrák jegyzéke

- 3.1. A KCMamba blokk teljes adatfolyam-diagramja. A sárga blokkok $(Q_{\text{net}}, R_{\text{net}}, K_{\text{net}})$ tanítják a Kalman-statisztikákat, a lila blokkban történik a párhuzamos prefix-scan GPU-n $O(\log T)$ szinkronizációval. A zöld blokkok a bemeneti és kimeneti projekciókat jelölik. 15
- 3.2. A Blelloch-féle párhuzamos prefix-scan vizualizációja 8 elemű szekvencián. Minden lépésben a *step*-nyi távolságú két elem kombinálódik. $\log_2 T = 3$ lépés után minden h_t tartalmazza az előzmény-összesítést. Ez teszi lehetővé a szekvenciális állapotfrissítés hatékony GPU-s párhuzamosítását. 19
- 3.3. A KCMamba verem (stack) felépítése. A bemenettől a kimenetig az input projekció, 2 KC-blokk LayerNorm rétegekkel, output normalizáció, majd cross-attention réteg követi egymást a lassú ($s = 4$) időskála integrálásához. 21
- 3.4. Szintetikus jel szűrése: **vanilla SSM** (standard Mamba, rögzített A) vs. **KCMamba** (adaptív K -gain) közepes ($\sigma = 2$) és erős ($\sigma = 5$) zajnál. *Felső sorok*: az előrejelzett jel a valódihoz képest. A KCMamba szorosabban követ zajban is. *Alsó sorok*: a tanult K -értékek időbeli változása. A vanilla SSM A -ja rögzített, a KCMamba A -ja adaptívan nő zajjal. 22
- 3.5. **Bal**: Impulzus-válasz függvény: az állapot exponenciális csökkenése $h_k = A^k$ a k késleltetés függvényében. A KCMamba A -értéke adaptívan nő a zajszinttel ($0.294 \rightarrow 0.926$), ezzel a memória-horizont is növekszik. **Jobb**: Effektív memória-horizont (az a lépésszám, ameddig $A^k > 5\%$) a zajszint függvényében. Az LSTM és vanilla SSM rögzített horizonttal rendelkeznek, a KCMamba zajban automatikusan hosszabb memóriát tart fenn. 22

- 3.6. A KCMamba alapú kereskedési rendszer hexagonális architektúrájának áttekintése. A **Core** csomag tartalmazza a TradingEngine-t (pipeline: állapotbecslés $\rightarrow$ predikció $\rightarrow$ szignál $\rightarrow$ kockázat $\rightarrow$ végrehajtás), a domain modelleket és az alkalmazásportokat. A külső adapterek (Infrastructure, Web, Testing, Offline Training, Observability, Analytics) kizárólag az interfészeken (**IEventBusPort**, **IBrokerGatewayPort** stb.) keresztül kommunikálnak a Core-ral. . . . 25
- 4.1. **Bal:** MSE a zajszint függvényében, Exchange/ $s = 16$ konfiguráció. A Fair Mamba a legjobb zajmentes MSE-t adja ($\sigma = 0$), de jelentős degradációt mutat $\sigma = 5$ -nél. A KCMamba degradációja +66% -nál stabilizálódik. **Jobb:** A KCMamba Kalman-gain (K), felejtési faktor ($A = 1 - K$) és zajbecslés (R) automatikus adaptációja a zajszint növekedésével. Magas zajnál K meredék esést mutat ($0,706 \rightarrow 0,073$). 29
- 4.2. MSE hőtérkép mind a négy modellre (ARIMA, LSTM, Fair Mamba, KCMamba), minden konfigurációra és zajszintre. A sötétpiros mezők magas hibát jeleznek, MSE > 4 . Az ARIMA és a Fair Mamba a $s=16$ konfigurációkban összességében $\sigma \geq 3$ -nál csődöt vallanak, míg a KCMamba végig stabil marad. 31
- 4.3. MSE-degradáció logaritmikus skálán ($\sigma = 0 \rightarrow \sigma = 5$) az összes konfigurációban. A Fair Mamba három esetben erős degradációt mutat ($> 10\,000\%$). A KCMamba (zöld) és az LSTM (kék) lényegesen stabilabbak. A kiemelten jelölt mező az Exchange/ $s = 16$ esetet jelöli. 31
- 4.4. Zajrobustussági összehasonlítás mind a négy modellre: MSE a zajszint (σ) függvényében, logaritmikus skálán. Az ARIMA (narancssárga, szaggatott) $s=16$ -nál katasztrofálisan összeomlik ($\sigma=5$ -nél MSE > 15). A Fair Mamba szintén instabillá válik $s=16$ -nál. A KCMamba és az LSTM végig mérsékelt tartományban maradnak. 32
- 4.5. Előrejelzés-vizualizáció három zajszinten (ETTh-típusú jel, $T = 96$ lookback, $H = 96$ horizon). Fekete = valós jel, zöld = KCMamba, narancssárga = Fair Mamba, kék = LSTM. **Bal:** $\sigma = 0$ -nál mindkét Mamba-modell pontosan követi a jelet. **Közép/Jobb:** A zaj növekedésével a KCMamba sima marad, a Fair Mamba instabillá válik, az LSTM lassan konvergál. 33

- 4.6. **Bal:** Paraméterszám vs. MSE ($\sigma=3$ átlag). A KCMamba egyszerre a legkisebb paraméterszámú és a legjobb MSE-jű, optimális Pareto-pozícióban van. **Jobb:** Epoch-tanítási idők (másodpercben). A PyTorch implementáció KCMamba-t $\approx 40\%$ -kal lassítja, de Triton-kernellel gyorsabb lesz az LSTM-nél. 33
- 4.7. **Bal:** Áteresztőképesség (minta/s) $T = 96$ -nál, kötegméret = 32, RTX 3060-on. A cuDNN-optimalizált LSTM a referencia; a natív PyTorch szelektív scan $3,4\times$ lassabb, egy Triton-kernelre írt KCMamba viszont LSTM-szintű sebességet ér el. **Jobb:** LSTM-hez normált relatív sebesség. A Triton-kernel rövid szekvenciákra szükséges, de nem elégséges optimalizáció. $T \geq 2000$ -nél a scan $O(\log T)$ mélysége várhatóan $2\text{--}10\times$ -es előnyt ad a szelektív SSM-ek irodalmában [2]. 34
- 4.8. MSE a szekvenciahossz (T) függvényében $\sigma=3$ (bal oszlop) és $\sigma=5$ (jobb oszlop) zajszinten, ETTm1 (felső sor) és Exchange (alsó sor) adatkészleteken. A KCMamba az egyetlen modell, amely mindkét adatkészleten és mindkét zajszinten javuló trendet mutat a szekvenciahossz növekedésével. 36
- 4.9. MSE a szekvenciahossz függvényében sűrű ($s=1$, felső sor) és ritka ($s=16$, alsó sor) ablakozásnál, $\sigma=3$ (bal oszlop) és $\sigma=5$ (jobb oszlop) zajszinten az Exchange adatkészleten. Az $s=16$ panelokon a tengely 4,0-nál el van vágva (a Fair Mamba MSE-je 8,1–8,5-re ugrik). A KCMamba minden konfigurációban stabil. A Fair Mamba $s=16$ -nál $\sigma \geq 3$ esetén katasztrofikusan összeomlik. 38